

PYTHON

PROYECTO FINAL

Ramón Márquez Fernández



RAMÓN MÁRQUEZ FERNÁNDEZ

Dashboard de RRHH y Gestión de Personal

RESPONSABLE DEL DESARROLLO DE LA APLICACIÓN

Fecha: 9 de octubre de 2026

ÍNDICE DE CONTENIDOS

1. DESCRIPCIÓN GENERAL DEL PROYECTO.....	7
1.1 Contexto	7
1.2 Problema que resuelve.....	7
1.3 Solución desarrollada	7
1.4 Usuarios y perfiles	8
1.5 Arquitectura	9
2. OBJETIVOS Y ALCANCE DEL PROYECTO	10
2.1 Objetivos	10
2.2 Alcance: qué incluye.....	10
2.3 Alcance: qué queda fuera.....	11
2.4 Relación con la propuesta aprobada.....	11
3. STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS	12
3.1 Stack tecnológico.....	12
3.2 Alternativas evaluadas	12
Framework del backend: Flask frente a Django y FastAPI	12
Base de datos: MariaDB en local frente a MySQL de producción.....	13
Frontend: Angular frente a plantillas Jinja con Plotly	13
Sesión: JWT frente a sesiones con cookie	13
Agregación: SQL frente a Pandas	13
4. MODELO DE DATOS.....	14
4.1 Tipo de base de datos.....	14
4.2 Esquema	15
4.3 Descripción de las tablas	16
4.4 La tabla de partes en detalle	16
4.5 Índices y rendimiento	17
4.6 Datos de prueba y protección de datos (LOPD)	17
5. REQUISITOS DE LA APLICACIÓN	19
5.1 Requisitos funcionales.....	19
RF-01. Inicio de sesión en dos pasos	19
RF-02. Control de accesos por perfil	20



RF-03. Mantenimiento y cierre de la sesión.....	24
RF-04. Resumen general de plantilla.....	24
RF-05. Directorio de personal.....	26
RF-06. Horas extra por periodos	28
RF-07. Horas extra por empleado	31
RF-08. Ranking de horas combo.....	31
RF-09. Total de servicios prestados (SP).....	32
RF-10. Formulario de parte de trabajo.....	33
RF-11. Mis partes.....	37
RF-12. Persistencia en la base de datos	38
RF-13. API REST documentada	39
5.2 Requisitos no funcionales.....	40
RNF-01. Seguridad.....	40
RNF-02. Rendimiento	41
RNF-03. Calidad y mantenibilidad	42
RNF-04. Portabilidad e instalación	42
RNF-05. Usabilidad	43
RNF-06. Protección de datos	43
6. MANUAL DE INSTALACIÓN	44
6.1 Requisitos previos	44
6.2 Obtener el proyecto y crear los ficheros .env	44
6.3 Arrancar la base de datos.....	45
6.4 Backend (Flask).....	45
6.5 Frontend (Angular)	46
6.6 Operaciones habituales y problemas frecuentes.....	46
7. METODOLOGÍA Y HERRAMIENTAS.....	47
7.1 Forma de trabajo	47
7.2 Uso de un asistente de inteligencia artificial.....	47
7.3 Herramientas.....	48
8. CONCLUSIONES	49
8.1 Conclusiones generales	49
8.2 Aprendizajes	49



8.3 Dificultades encontradas.....	49
9. EVOLUTIVOS DEL PROYECTO.....	51
9.1 Mejoras funcionales	51
Circuito de firmas desde el dashboard (prioridad alta).....	51
Indicadores de absentismo y rotación (prioridad media)	51
Exportación de informes (prioridad media)	51
Partes del equipo para los mandos (prioridad baja)	51
9.2 Mejoras técnicas.....	51
9.3 Hoja de ruta.....	52

ÍNDICE DE FIGURAS

Figura 1. Arquitectura de la aplicación: frontend Angular, API Flask y base de datos compartida con la aplicación Java.	9
Figura 2. Esquema de la base de datos (editable en diagrams.net: docs/modelo_datos.drawio).	15
Figura 3. Tablas de la base de datos e partes_local y volumen de los datos ficticios cargados.	18
Figura 4. Pantalla de inicio de sesión (paso 1: correo y contraseña).	19
Figura 5. Credenciales incorrectas: mensaje genérico.	20
Figura 6. Usuario con varios perfiles: selector con RRHH preseleccionado.	20
Figura 7. Menú de RRHH: todas las pantallas.	21
Figura 8. Menú de un mando: sin restricciones de pantalla, pero con datos limitados a sus grupos.	21
Figura 9. Menú de un empleado: sus partes y sus horas.	22
Figura 10. Perfiles en la tabla zgrroles: de aquí salen los permisos de cada usuario.	22
Figura 11. Un mando del grupo «Recogida Norte - Equipo 1» solo ve a los trabajadores de su grupo.	23
Figura 12. Un empleado solo ve sus propias horas extra.	23
Figura 13. Usuario conectado, perfil activo y opción de cerrar sesión.	24
Figura 14. Resumen general de plantilla (RRHH).	25
Figura 15. Resumen filtrado por un grupo.	25
Figura 16. Directorio de personal visto por RRHH.	26
Figura 17. Búsqueda de empleados por texto.	27
Figura 18. El mismo directorio visto por un mando: solo su grupo.	27
Figura 19. Horas extra por periodos del ejercicio 2025 (RRHH), con indicadores, tablas y gráficos.	29
Figura 20. Filtrado por un periodo de nómina concreto.	30
Figura 21. Modo «año natural»: se agrupa por la fecha del parte.	30
Figura 22. Horas extra por empleado (ejercicio 2025).	31
Figura 23. Ranking de horas combo y compensables.	32
Figura 24. Servicios prestados de los últimos 12 y 24 meses.	33
Figura 25. Formulario con 25 h extra y sin motivos: total en rojo, mensajes junto a cada campo y casillas festivas bloqueadas en día laborable.	35
Figura 26. Formulario completo y válido: presencia calculada (11 h), horas con su motivo y marca SP.	36
Figura 27. El servidor rechaza el mismo parte aunque se envíe sin pasar por el formulario.	37
Figura 28. Mis partes: el parte recién guardado (editable) y los ya validados (solo consulta).	37
Figura 29. Un parte ya validado se abre en modo consulta.	38
Figura 30. Fila creada en zparte por el formulario: periodo de nómina, estado B, presencia 110000, horas, motivos y marcas.	39
Figura 31. Documentación Swagger de la API.	40
Figura 32. Pruebas reales contra la API: credenciales erróneas (401), sin token (401), perfil sin permiso (403) y parte de otro empleado (404).	41
Figura 33. Contraseñas guardadas como hash bcrypt. En los datos de demostración todos los usuarios comparten la misma contraseña, por eso coincide el hash.	41



Figura 34. Ejecución de los tests del backend: 159 pasados.....	42
Figura 35. Ejecución de los tests del frontend: 32 pasados.	42
Figura 36. Contenedor de MariaDB en marcha y saludable.	45



1. DESCRIPCIÓN GENERAL DEL PROYECTO

1.1 Contexto

En la empresa, los trabajadores registran cada día un **parte de trabajo** en una aplicación Java corporativa (e-Partes): el turno realizado, las horas de presencia, las horas extra de cada tipo, las llamadas atendidas y una serie de marcas (sustituciones, trabajos de superior categoría, teletrabajo...). Cada parte sigue un circuito de firmas: lo crea el empleado o su encargado, recibe el visto bueno del jefe de área, lo firma el jefe de departamento y, finalmente, Recursos Humanos lo traspasa a nómina.

Toda esa información vive en una base de datos MySQL compartida (tablas `zparte`, `userpayroll`, `zgrroles`, `zperiodos`, `eppartstatus` y catálogos). La aplicación Java está orientada a grabar y firmar partes, pero no ofrece una visión agregada: un responsable no puede ver de un vistazo cuántas horas extra acumula su equipo, qué departamentos concentran más horas o cómo evolucionan los servicios prestados mes a mes.

1.2 Problema que resuelve

- **Falta de analítica por perfil:** RRHH necesita la visión global de la plantilla, cada mando solo la de sus grupos y cada empleado la suya, y hoy esa información se obtiene con consultas manuales.
- **Datos dispersos en un esquema heredado:** la tabla de partes tiene 102 columnas con nombres abreviados (DL, DFC, NLCOP...) que solo conocen los técnicos.
- **Registro del parte poco guiado:** no hay controles que impidan, por ejemplo, declarar más horas extra que horas de presencia o dejar sin motivo un bloque de horas.

1.3 Solución desarrollada

El proyecto es una **aplicación web** formada por una API REST en **Python con Flask** y un frontend en **Angular**. Se conecta directamente a la misma base de datos que la aplicación Java, sin réplicas ni procesos ETL, respetando su esquema (no se renombra ninguna tabla ni columna). Sus funciones principales son:

- **Inicio de sesión seguro** con las credenciales corporativas (contraseñas con hash bcrypt) y sesión mediante tokens JWT.
- **Control de accesos por perfil** (RRHH, mando y empleado) tomado de la tabla `zgrroles`; el filtrado de datos se aplica siempre en el servidor.
- **Cuadro de mando** con resumen de plantilla, directorio de personal, horas extra por periodos y por empleado, ranking de horas combo y servicios prestados de los últimos 12 y 24 meses.



- **Formulario de parte de trabajo** para todos los empleados, estructurado por bloques de horas y con 13 controles lógicos, y una pantalla «Mis partes» para consultarlos, corregirlos o borrarlos mientras no estén validados.
- **API documentada con Swagger**, instalación con Docker, datos de demostración ficticios y una batería de 191 tests automáticos.

1.4 Usuarios y perfiles

Perfil	Origen en la base de datos	Qué datos ve
RRHH (HR)	Fila con ROLNAME = 'HR' en zgrroles	Todos los partes y toda la plantilla.
Mando (VB / FI)	Filas con ROLNAME = 'VB' o 'FI'	Los partes de los grupos (CODGR) en los que tiene ese rol, más los suyos propios.
Empleado	Cualquier usuario activo (sin rol o con otros)	Únicamente sus propios partes.

Un usuario puede tener varios perfiles (por ejemplo, RRHH y VB): al iniciar sesión elige con cuál entra, el servidor comprueba que realmente lo posee y se preselecciona el de mayor prioridad (RRHH > mando > empleado).

1.5 Arquitectura

La aplicación sigue una arquitectura de tres capas. El navegador ejecuta una aplicación de página única (SPA) que solo dibuja y valida en vivo; toda la lógica de negocio y de seguridad está en el backend, organizado en rutas finas, una capa de servicios orientada a objetos y modelos ORM.

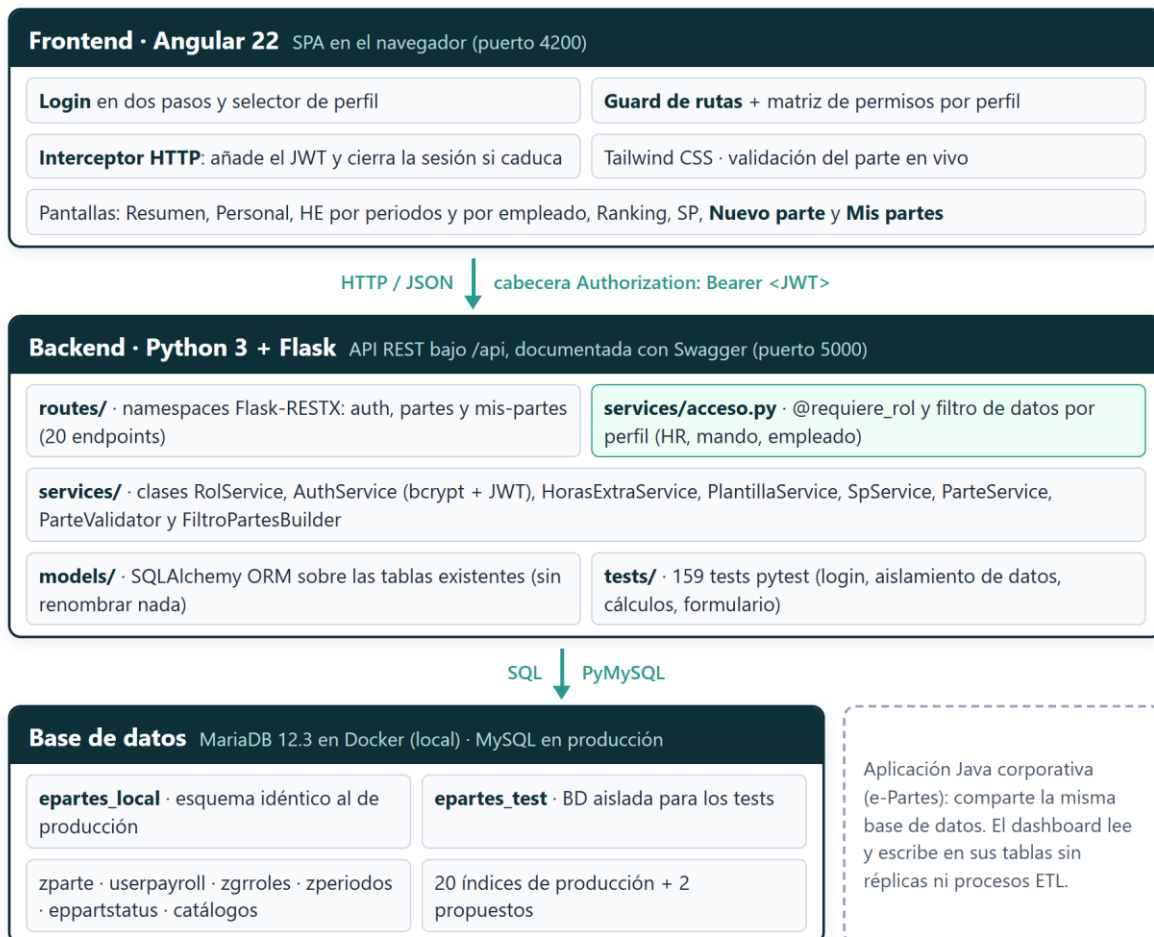


Figura 1. Arquitectura de la aplicación: frontend Angular, API Flask y base de datos compartida con la aplicación Java.

2. OBJETIVOS Y ALCANCE DEL PROYECTO

2.1 Objetivos

Los objetivos se han definido de forma que se puedan comprobar con una prueba o una medición concreta:

N.º	Objetivo	Cómo se mide	Resultado
O1	Control de accesos real por perfil, aplicado en el servidor.	Tests de aislamiento de datos para RRHH, VB, FI, empleado y usuario con varios perfiles.	Cumplido (34 tests de aislamiento).
O2	Contraseñas nunca en texto plano y sesión segura.	Hash bcrypt en userpayroll; JWT firmado con caducidad de 8 h.	Cumplido.
O3	Pantallas del cuadro de mando rápidas con un volumen realista.	Respuesta de la API con ~100.000 partes.	Cumplido: $\leq 0,35$ s por petición.
O4	Registro del parte con controles lógicos.	13 reglas validadas en navegador y servidor; 30 tests del validador.	Cumplido.
O5	Instalación reproducible desde cero.	Instalación en una carpeta limpia siguiendo solo el manual.	Cumplido (Docker + manual verificado).
O6	Calidad verificable.	Tests automáticos de backend y frontend.	159 tests pytest y 32 tests Angular en verde.
O7	Respeto a la LOPD.	Ningún dato real en el proyecto ni en la memoria.	Cumplido: datos 100 % ficticios.

2.2 Alcance: qué incluye

- Autenticación con las credenciales de `userpayroll` y elección de perfil según `zgrroles`.
- Filtrado de datos por perfil en todos los endpoints de consulta.
- Pantallas: Resumen General, Personal, HE por periodos, HE por empleado, Ranking HE Combo, Total SP retribuidas, Nuevo parte y Mis partes.
- Alta, consulta, edición y borrado de los partes propios mientras estén en estado B (sin visto bueno).
- API REST documentada en Swagger, instalación con Docker Compose, generador de datos de demostración y medición de rendimiento de las consultas con distintos conjuntos de índices.



2.3 Alcance: qué queda fuera

- El circuito de firmas (visto bueno, firma y traspaso a nómina) sigue haciéndose en la aplicación Java; el dashboard muestra el estado pero no lo cambia.
- Los indicadores de absentismo y rotación citados en la propuesta inicial (ver evolutivos).
- El alta y baja de empleados, grupos o roles, que son responsabilidad de la aplicación corporativa.
- El despliegue en un servidor de producción con HTTPS.

2.4 Relación con la propuesta aprobada

Elemento de la propuesta	Estado	Comentario
Perfiles Administrador / Mando / Empleado (RBAC)	Cumplido	El perfil administrador corresponde a RRHH (ROLNAME = HR).
Integración en caliente con la BD de la app Java	Cumplido	Mismo esquema, sin renombrar tablas ni columnas.
Módulo 1: control de asistencia y flujo diario	Cumplido	Formulario de parte con cálculo de presencia y «Mis partes».
Módulo 2: panel de KPIs con gráficos	Parcial	Horas extra, plantilla y SP con gráficos; absentismo y rotación quedan como evolutivo.
Módulo 3: gestión operativa de empleados	Cumplido	Directorio de personal y HE por empleado con filtros por fechas, grupo, área y estado.
Índices compuestos y rendimiento	Cumplido	Medición con 3 escenarios de índices y 2 índices propuestos.
Pandas / Plotly para gráficos	Sustituido	Los gráficos se dibujan en Angular; la agregación se hace en SQL (ver apartado 3).

3. STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS

3.1 Stack tecnológico

Capa	Tecnología	Uso en el proyecto
Lenguaje backend	Python 3.13	Toda la lógica de negocio, la API, los scripts de datos y los tests.
Framework web	Flask 3.0 + Flask-RESTX 1.3	API REST organizada en namespaces y documentación Swagger automática en /api/doc.
Acceso a datos	SQLAlchemy 2.0 (Flask-SQLAlchemy) + PyMySQL	Modelos ORM sobre las tablas existentes y consultas agregadas.
Serialización	marshmallow / flask-marshmallow	Conversión de partes a JSON en el listado.
Seguridad	bcrypt + flask-jwt-extended	Hash de contraseñas y tokens JWT con el perfil activo y los grupos del mando.
Base de datos	MariaDB 12.3 (local) / MySQL (producción)	Esquema compartido con la aplicación Java; utf8mb3 como en producción.
Contenedores	Docker + Docker Compose	Arranque de la base de datos con el volcado y la base de datos de tests.
Frontend	Angular 22 + TypeScript + Tailwind CSS	SPA con login, guard de rutas, interceptor HTTP y formularios reactivos.
Pruebas	pytest, Vitest (ng test), Playwright	Tests de API y servicios, tests de componentes y recorridos en navegador.
Control de versiones	Git + GitHub	Historial por fases y repositorio público.

3.2 Alternativas evaluadas

Framework del backend: Flask frente a Django y FastAPI

Django aporta mucho de serie (ORM, administración, plantillas), pero su ORM y sus migraciones están pensados para que la aplicación sea dueña del esquema, y aquí el esquema pertenece a la aplicación Java y no se puede modificar. **FastAPI** ofrece validación con tipos y alto rendimiento asíncrono, pero el acceso a MySQL es síncrono y el curso se centra en Flask. **Flask** se eligió por ser ligero, por encajar con un esquema heredado y porque, con **Flask-RESTX**, genera la documentación Swagger sin trabajo adicional.



Base de datos: MariaDB en local frente a MySQL de producción

Producción usa MySQL. Para el entorno local se eligió **MariaDB 12.3** en Docker porque el volcado disponible procede de MariaDB, ambos motores comparten dialecto SQL y el driver es el mismo (PyMySQL). Para que las mediciones sean representativas se replicó el esquema real: misma codificación (`utf8mb3_general_ci`) y los mismos 20 índices secundarios de la tabla de partes. Se descartó SQLite porque no reproduce el comportamiento de índices ni de transacciones del motor real.

Frontend: Angular frente a plantillas Jinja con Plotly

La propuesta inicial contemplaba plantillas de servidor con Plotly. Se optó por una **SPA en Angular** porque el cuadro de mando tiene muchos filtros interdependientes (año, periodo, departamento, trabajador, estado) y un formulario con validación en vivo, algo más mantenible con componentes y tipado. Además, separar frontend y API permite que la misma API la consuman otras herramientas. La desventaja es que hay que instalar Node.js, lo que se resuelve en el manual de instalación.

Sesión: JWT frente a sesiones con cookie

Con un frontend separado, los **tokens JWT** permiten una API sin estado: el token lleva firmado el perfil activo y los grupos del mando, y el servidor lo verifica en cada petición. La alternativa (Flask-Login con cookie de sesión) obligaría a gestionar CSRF y sesiones en el servidor. Como contrapartida, un cambio de roles no se aplica hasta el siguiente inicio de sesión (máximo 8 horas); se documenta como limitación.

Agregación: SQL frente a Pandas

Todos los totales (horas por mes, por departamento, por trabajador...) se calculan con `GROUP BY` en la base de datos. Cargar 100.000 partes en Pandas para agregarlos en Python sería más lento y consumiría más memoria; agregando en SQL solo viajan por la red los resultados.



4. MODELO DE DATOS

4.1 Tipo de base de datos

Se trabaja sobre una **base de datos relacional** (MySQL en producción, MariaDB en local) con motor InnoDB. La elección viene dada por el proyecto: la aplicación Java ya usa este esquema y el objetivo es leerlo «en caliente». Una base de datos relacional es además la opción natural para estos datos: tienen una estructura fija, relaciones claras (empleado, grupo, periodo, estado) y necesitan agregaciones con `GROUP BY` y filtros por rangos de fechas, que se benefician de índices.

Una restricción importante es que **el esquema no pertenece a este proyecto**: no se puede renombrar ninguna tabla ni columna, ni añadir claves ajenas. Por eso las relaciones entre tablas son lógicas (por valor) y los modelos de SQLAlchemy se adaptan a los nombres existentes.



4.2 Esquema

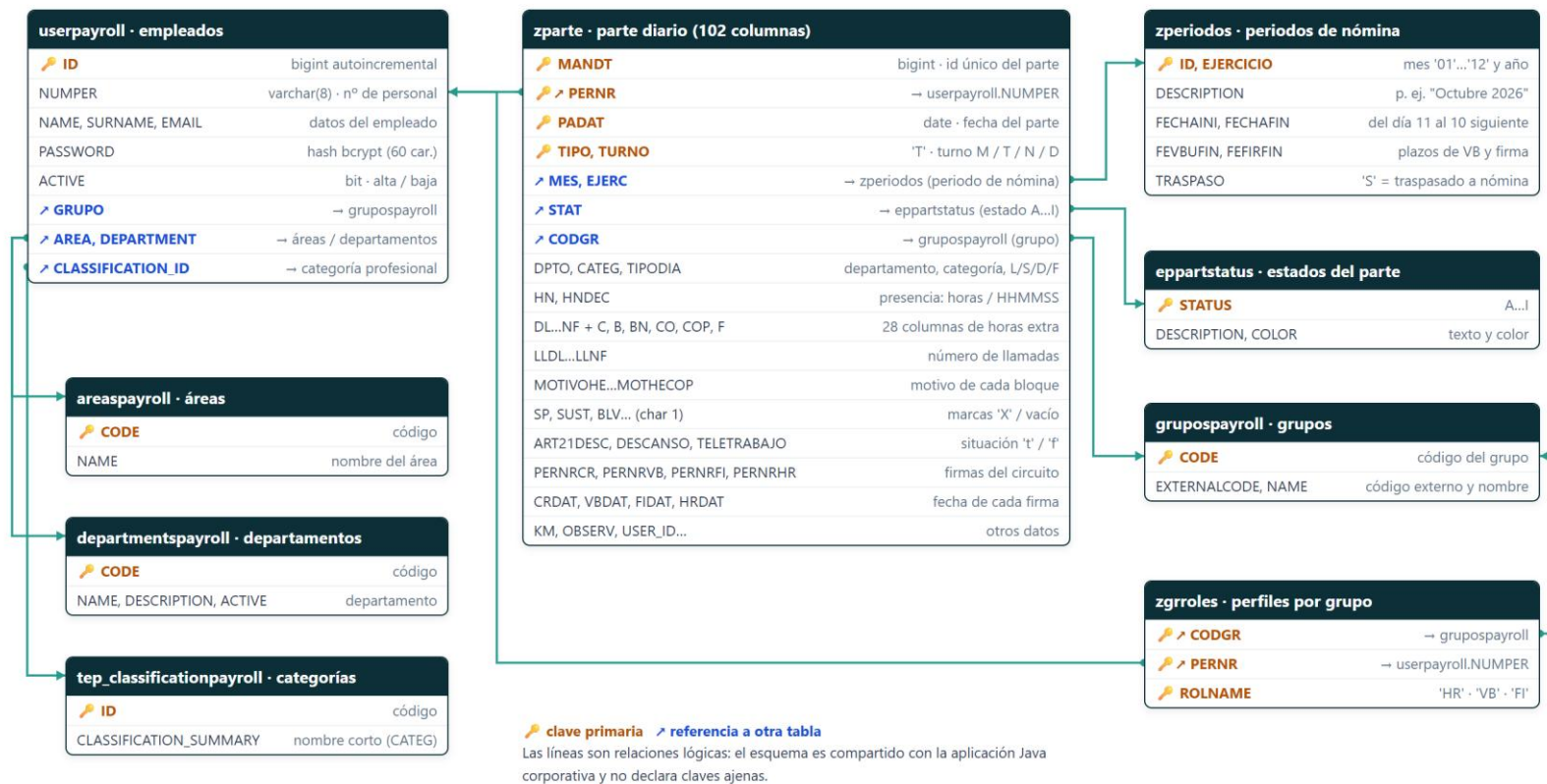


Figura 2. Esquema de la base de datos (editable en [diagrams.net: docs/modelo_datos.drawio](https://diagrams.net/docs/modelo_datos.drawio)).

4.3 Descripción de las tablas

Tabla	Contenido	Clave primaria	Filas (demo)
zparte	Parte diario de cada trabajador: turno, presencia, horas extra, llamadas, marcas, motivos y firmas.	MANDT, PERNR, PADAT, TIPO, TURNO	99.908
userpayroll	Empleados: número de personal, nombre, correo, contraseña (hash), alta/baja, grupo, área y departamento.	ID	250
zgrroles	Perfiles de cada persona en cada grupo: HR, VB o FI.	CODGR, PERNR, ROLNAME	66
zperiodos	Periodos de nómina (del día 11 de un mes al 10 del siguiente) y si están traspasados.	ID, EJERCICIO	34
eppartstatus	Estados del parte, identificados con las letras A–I, con su descripción y color.	STATUS	9
grupospayroll	Grupos de trabajo.	CODE	25
areaspayroll / departmentspayroll	Áreas y departamentos de la empresa.	CODE	6 / 12
tep_classificationpayroll	Categorías profesionales.	ID	—

4.4 La tabla de partes en detalle

La tabla zparte tiene 102 columnas. Para el proyecto se han agrupado por bloques, y esa agrupación es la que usan tanto el formulario como los cálculos de horas extra:

Bloque	Columnas	Significado
Identificación	MANDT, PERNR, PADAT, TIPO, TURNO	Identificador único, trabajador, fecha, tipo y turno (M, T, N, D).
Periodo y estado	MES, EJERC, STAT, TIPODIA	Periodo de nómina, estado del circuito y tipo de día (L, S, D, F).
Presencia	HN, HNDEC	Horas de presencia (entero y formato HHMMSS).
Horas extra	DL, DF, NL, NF + terminaciones C, B, BN, CO, COP, F	Tramo (diurna/nocturna, laborable/festiva) × tipo: normales, compensar, busca, busca no pagada, combo, combo programadas y F.
Llamadas	LLDL, LLDF, LLNL, LLNF	Número de llamadas atendidas por tramo.

Bloque	Columnas	Significado
Motivos	MOTIVOHE, MOTIVOHEC, MOTIVOHEB, MOTHEBNP, MOTHECO, MOTHECOP, MOTIVOHFM, MOTIVOLLA	Motivo obligatorio de cada bloque con horas.
Marcas	SP, SUST, CP, PP, B, D, DD, P52, P60, BLV, BSDF, CPT, PN	Marcas de casilla: 'X' o vacío.
Situación del día	ART21DESC, DESCANSO, TELETRABAJO	Solo una puede valer 't'.
Firmas	PERNRRCR, PERNRVB, PERNRFI, PERNRHR y sus fechas	Quién creó, dio el visto bueno, firmó y traspasó el parte.

4.5 Índices y rendimiento

La tabla de partes tiene en producción 20 índices secundarios. Con el script `medir_rendimiento.py` se midieron las consultas reales del dashboard (54 sentencias para los perfiles RRHH, mando y empleado) en tres escenarios. Los resultados completos, con el EXPLAIN de cada sentencia, están en `docs/rendimiento.md`:

Escenario	Tiempo total	Mejora
1. Solo clave primaria	3.867 ms	—
2. Índices de producción (20)	2.094 ms	–46 % respecto al escenario 1
3. Producción + 2 índices propuestos	1.572 ms	–25 % respecto a producción

Los dos índices propuestos son (`CODGR`, `PADAT`), que reduce la consulta por año natural de un mando de 221 ms a 48 ms, y (`SP`, `PADAT`), que reduce la pantalla de servicios prestados de 166 ms a 25 ms. Como el esquema es compartido, no se aplican directamente: quedan propuestos en `docs/indices_propuestos.sql` para el equipo de la aplicación corporativa.

4.6 Datos de prueba y protección de datos (LOPD)

El proyecto **no contiene ningún dato real**. Se dispuso de una exportación de producción, pero por la LOPD solo se usó para obtener estadísticas agregadas (proporción de turnos, de tipos de día, de partes con horas extra, periodos de nómina, etc.), sin copiar ni mostrar ningún registro. Con esas proporciones, el script `generar_datos.py` crea datos ficticios con semilla fija: 250 empleados inventados en 25 grupos y unos 100.000 partes desde enero de 2024.

```
Tablas y volumen de datos en epartes_local

MariaDB [epartes_local]> SHOW TABLES; SELECT ... COUNT(*) ...
Tables_in_epartes_local
areaspayroll
departmentspayroll
eppartstatus
grupospayroll
tep_classificationpayroll
userpayroll
zgrroles
zparte
zperiodos
tabla  filas
zparte 99907
userpayroll 250
zgrroles 66
zperiodos 34
grupospayroll 25
```

Figura 3. Tablas de la base de datos epartes_local y volumen de los datos ficticios cargados.

5. REQUISITOS DE LA APLICACIÓN

Las capturas se han tomado con la aplicación en funcionamiento y los datos ficticios de demostración. Los usuarios de prueba son `atorres@empresa.local` (RRHH), `eruiz@empresa.local` (RRHH y VB), `mgarcia@empresa.local` (VB), `crodriguez@empresa.local` (FI) y `jperez@empresa.local` (empleado), todos con la contraseña `Tirme2026!`.

5.1 Requisitos funcionales

RF-01. Inicio de sesión en dos pasos

- El usuario introduce su correo corporativo y su contraseña. El servidor busca el empleado por correo (sin distinguir mayúsculas), comprueba que esté activo y verifica la contraseña contra el hash bcrypt.
- Si el correo no existe o la contraseña es incorrecta, el mensaje es siempre el mismo («Correo o contraseña incorrectos»), para no revelar qué correos están registrados.
- Si el usuario tiene un solo perfil, entra directamente; si tiene varios, se muestra un selector con el de mayor prioridad preseleccionado. El servidor comprueba que el perfil elegido le pertenece.
- Al entrar, el servidor devuelve un token JWT con el número de personal, el perfil activo y los grupos del mando.

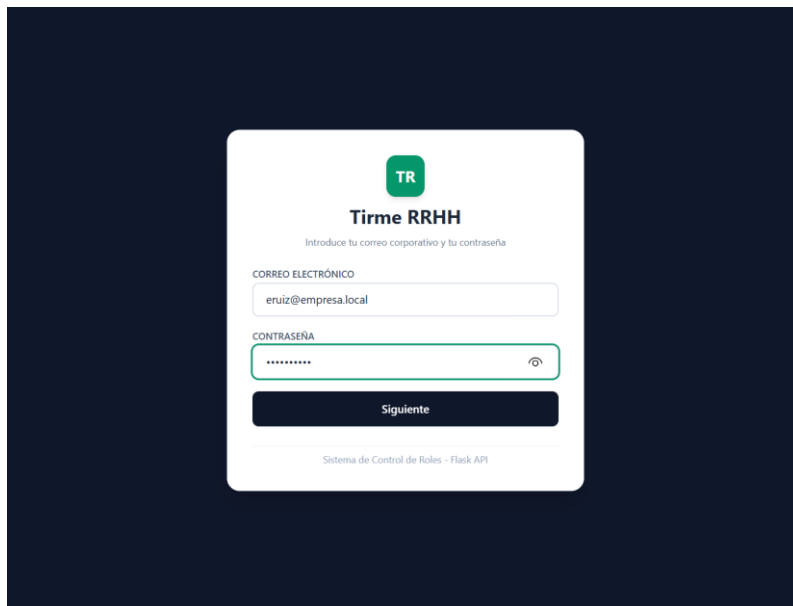


Figura 4. Pantalla de inicio de sesión (paso 1: correo y contraseña).

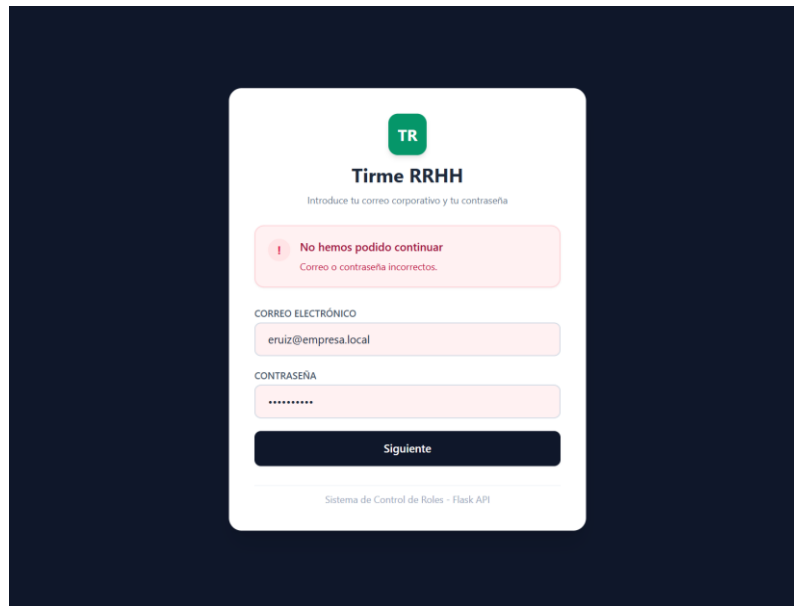


Figura 5. Credenciales incorrectas: mensaje genérico.

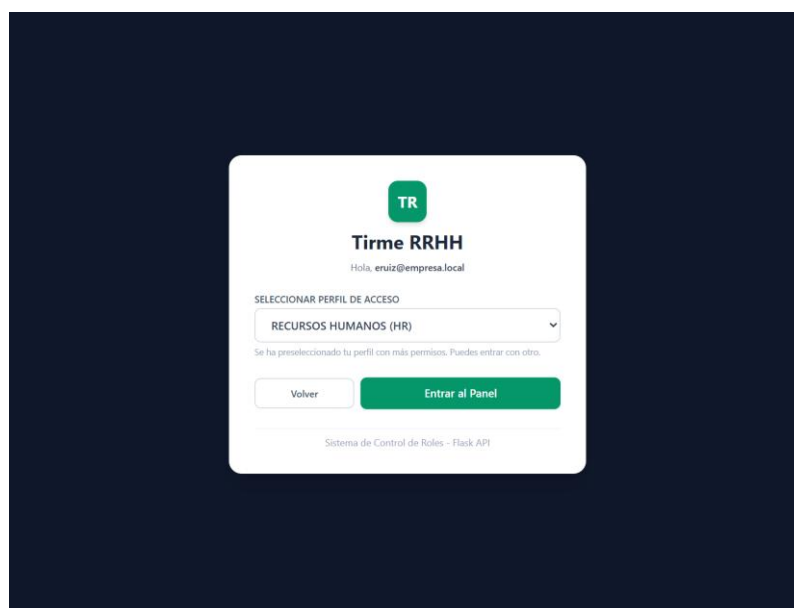


Figura 6. Usuario con varios perfiles: selector con RRHH preseleccionado.

RF-02. Control de accesos por perfil

- Los perfiles se leen de `zgrro1es`: HR da acceso total; VB o FI dan acceso a los grupos donde se tiene ese rol más los partes propios; cualquier usuario es, como mínimo, empleado.
- El menú lateral solo muestra las pantallas permitidas y un guard impide entrar escribiendo la URL.

- El filtrado se aplica **en el servidor**: todas las consultas pasan por un filtro de perfil central (`services/acceso.py`) y el decorador `@requiere_ro1` responde 403 si el perfil no tiene acceso.

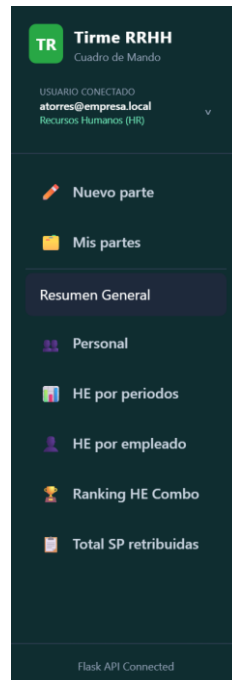


Figura 7. Menú de RRHH: todas las pantallas.

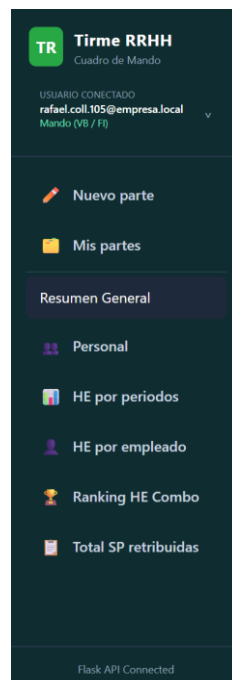


Figura 8. Menú de un mando: sin restricciones de pantalla, pero con datos limitados a sus grupos.

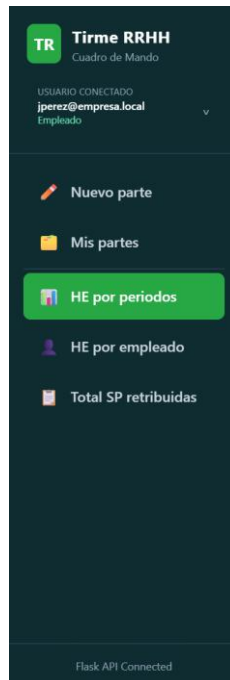


Figura 9. Menú de un empleado: sus partes y sus horas.

```
Roles en zgrroles (grupos 1 a 3)

MariaDB [epartes_local]> SELECT r.CODGR, g.NAME, r.PERNR, ..., r.ROLNAME FROM zgrroles r ...
CODGR  grupo  PERNR  empleado  ROLNAME
1      Grupo Técnico A 00000041  Marta Ramos Muñoz  FI
1      Grupo Técnico A 00000004  Ana Torres Vidal   HR
1      Grupo Técnico A 00000211  Rubén Martín Ramírez HR
1      Grupo Técnico A 00000002  María García López  VB
2      Grupo Operativo B 00000003  Carlos Rodríguez Silva FI
2      Grupo Operativo B 00000005  Elena Ruiz Martín   HR
2      Grupo Operativo B 00000211  Rubén Martín Ramírez HR
2      Grupo Operativo B 00000005  Elena Ruiz Martín   VB
3      Recogida Norte - Equipo 1 00000163  Javier Gil Martínez FI
3      Recogida Norte - Equipo 1 00000105  Rafael Coll Gutiérrez FI
3      Recogida Norte - Equipo 1 00000211  Rubén Martín Ramírez HR
```

Figura 10. Perfiles en la tabla zgrroles: de aquí salen los permisos de cada usuario.

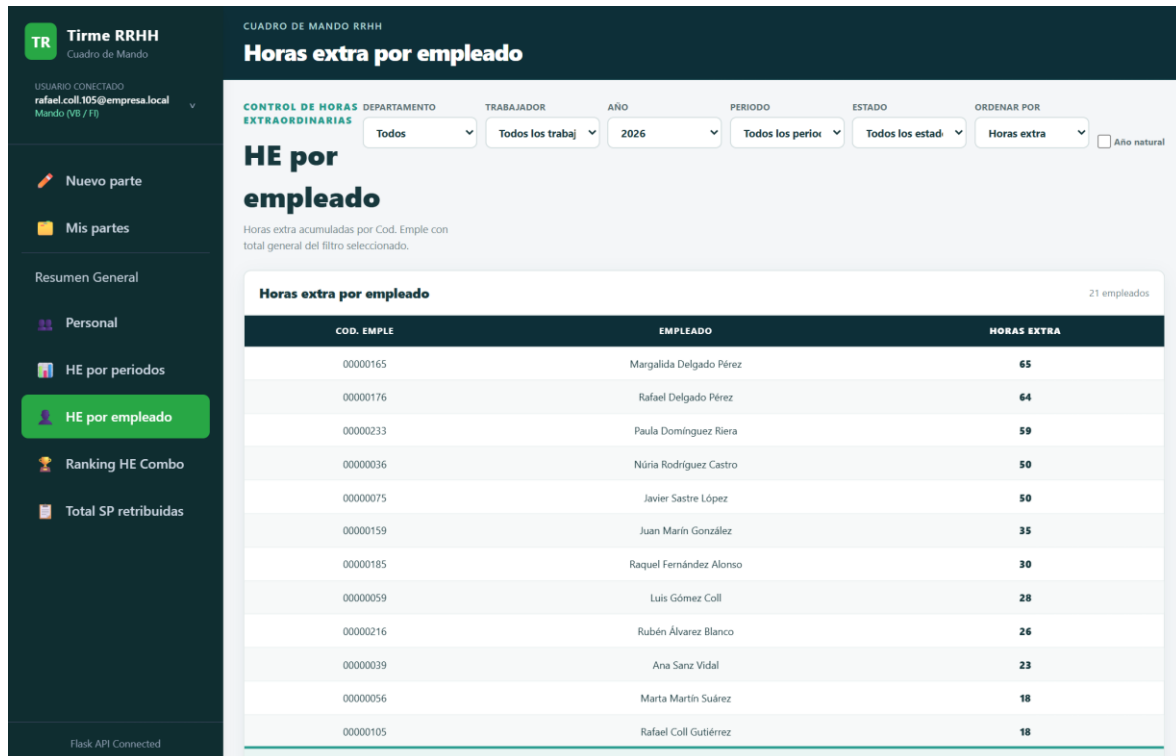


Figura 11. Un mando del grupo «Recogida Norte - Equipo 1» solo ve a los trabajadores de su grupo.

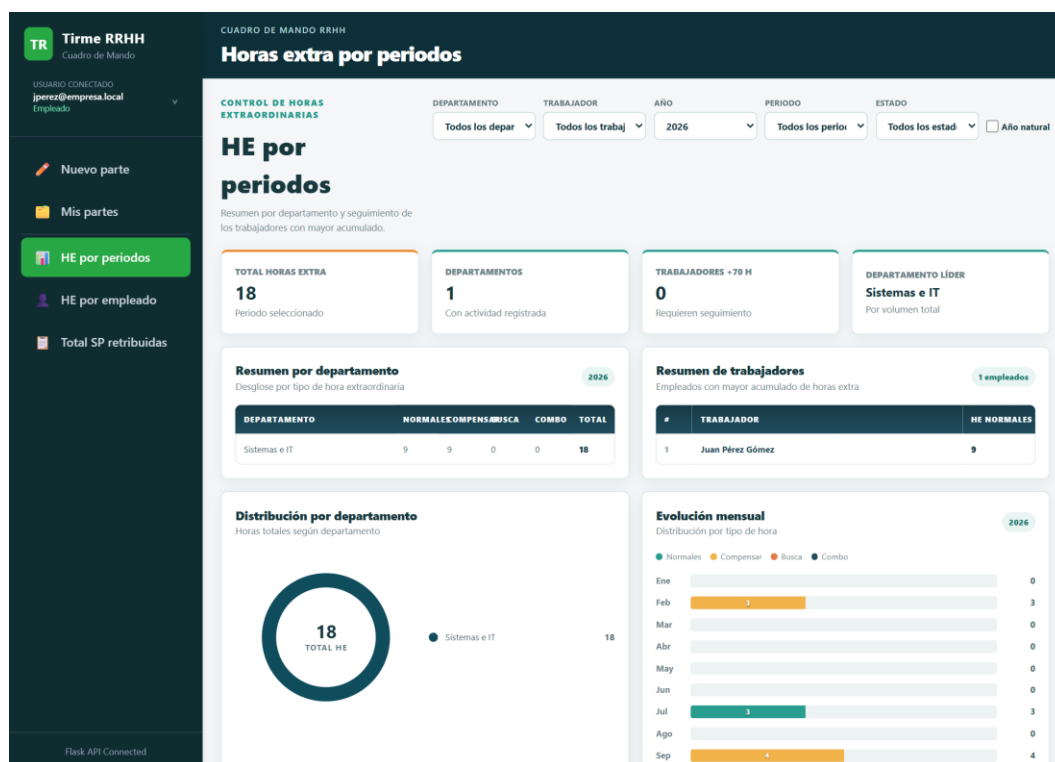


Figura 12. Un empleado solo ve sus propias horas extra.

RF-03. Mantenimiento y cierre de la sesión

- El token se guarda en el navegador; al recargar la página la sesión se mantiene mientras el token sea válido (8 horas).
- Si el servidor responde 401 (token caducado o manipulado), el interceptor HTTP cierra la sesión y vuelve al login.
- El usuario puede cerrar sesión desde el menú de su nombre.

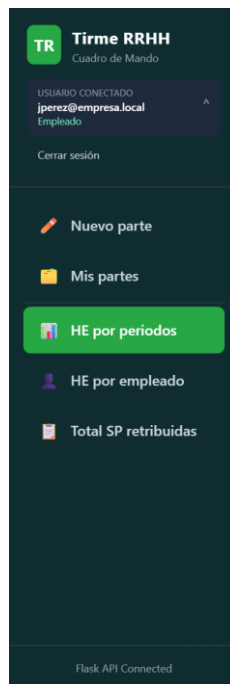


Figura 13. Usuario conectado, perfil activo y opción de cerrar sesión.

RF-04. Resumen general de plantilla

- Muestra el total de empleados activos y su desglose por grupo, área y departamento.
- Se puede filtrar por grupo y por área; las opciones de los filtros son solo las visibles para el perfil.
- Disponible para RRHH y mandos (un mando solo cuenta a la plantilla de sus grupos).

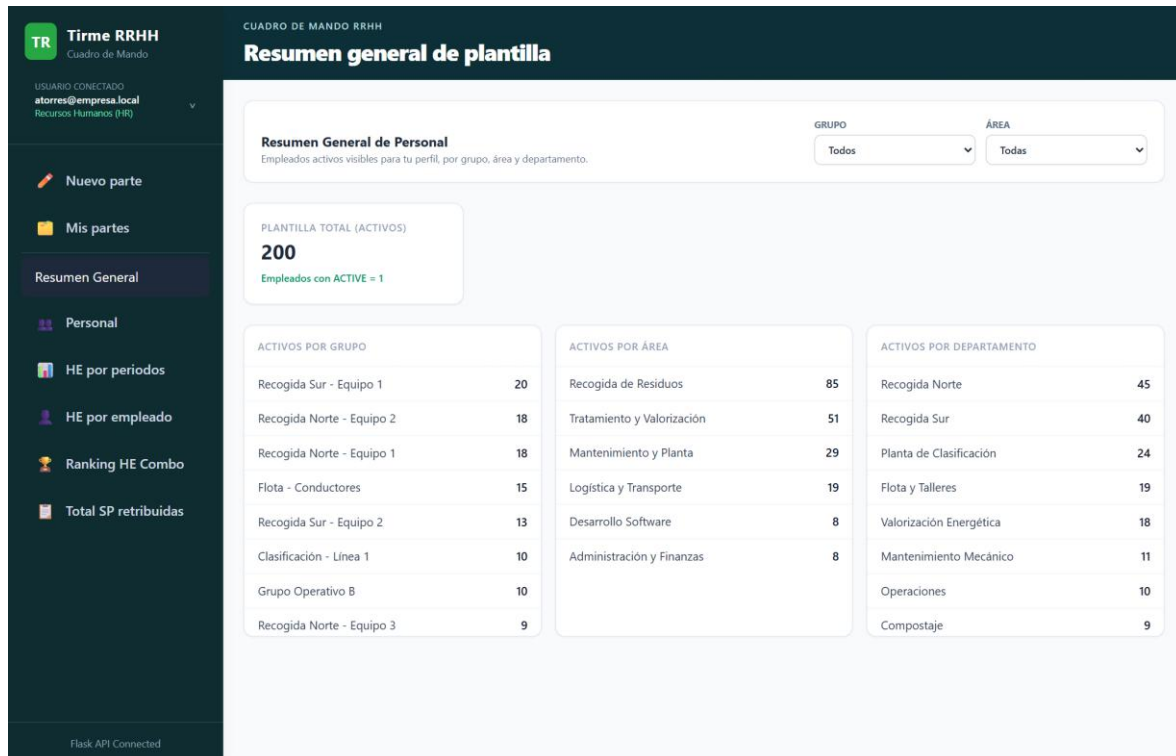


Figura 14. Resumen general de plantilla (RRHH).

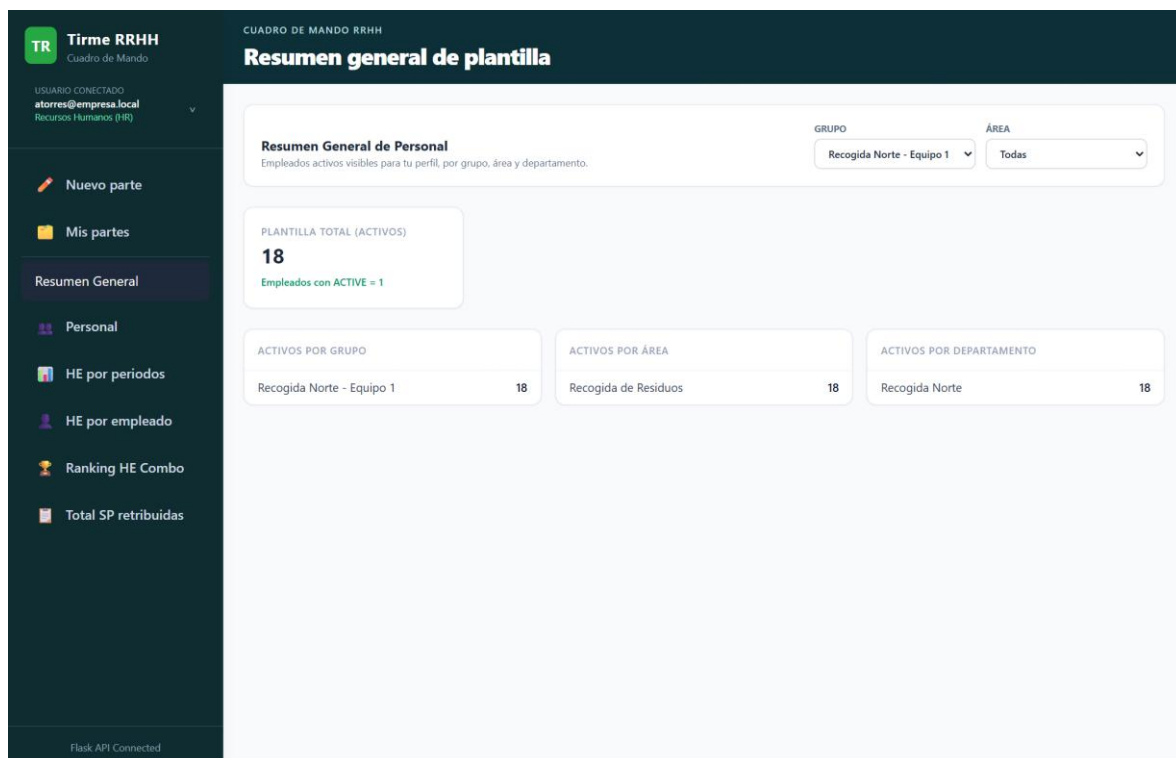
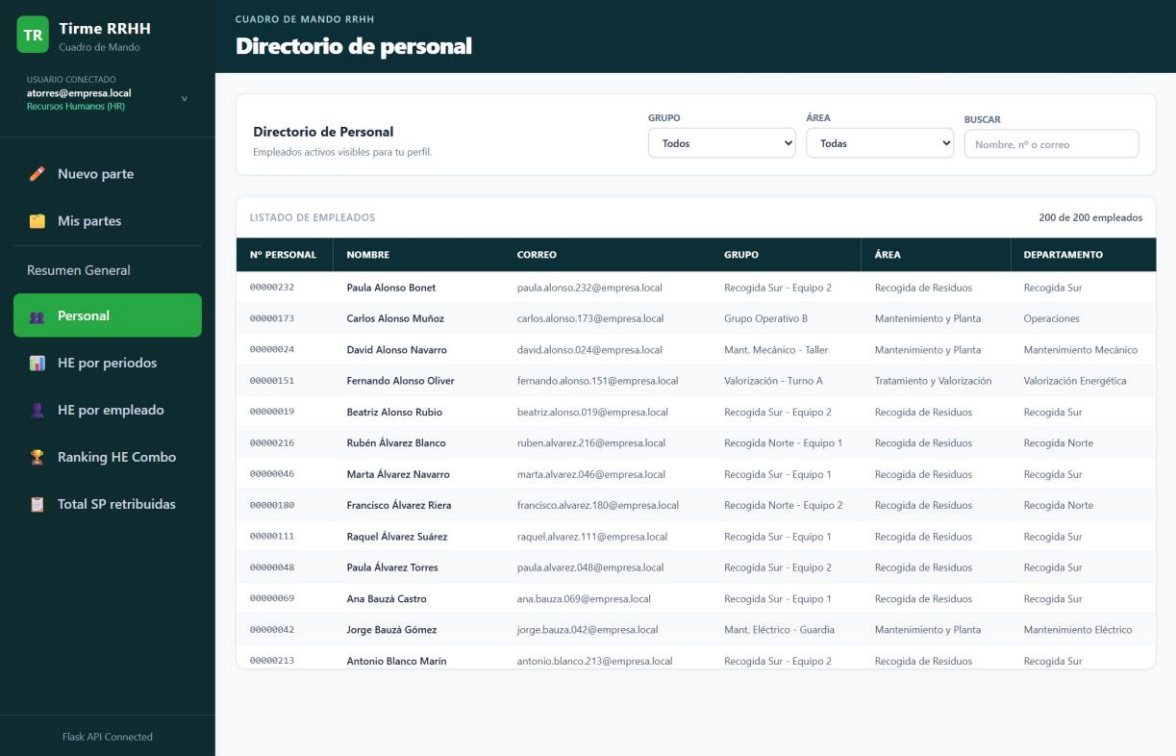


Figura 15. Resumen filtrado por un grupo.

RF-05. Directorio de personal

- Lista los empleados activos con número de personal, nombre, correo, grupo, área y departamento.
- Filtros por grupo y área y buscador de texto (nombre, número o correo).
- Nunca devuelve contraseñas, tokens ni el documento de identidad (lista blanca de columnas).



Tirme RRHH
Cuadro de Mando

USUARIO CONECTADO
atorres@empresa.local
Recursos Humanos (RRH)

Directorio de personal
Empleados activos visibles para tu perfil.

GRUPO: Todos | ÁREA: Todas | BUSCAR: Nombre, n° o correo

LISTADO DE EMPLEADOS 200 de 200 empleados

N° PERSONAL	NOMBRE	CORREO	GRUPO	ÁREA	DEPARTAMENTO
00000232	Paula Alonso Bonet	paula.alonso.232@empresa.local	Recogida Sur - Equipo 2	Recogida de Residuos	Recogida Sur
00000173	Carlos Alonso Muñoz	carlos.alonso.173@empresa.local	Grupo Operativo B	Mantenimiento y Planta	Operaciones
00000024	David Alonso Navarro	david.alonso.024@empresa.local	Mant. Mecánico - Taller	Mantenimiento y Planta	Mantenimiento Mecánico
00000151	Fernando Alonso Oliver	fernando.alonso.151@empresa.local	Valorización - Turno A	Tratamiento y Valorización	Valorización Energética
00000019	Beatriz Alonso Rubio	beatriz.alonso.019@empresa.local	Recogida Sur - Equipo 2	Recogida de Residuos	Recogida Sur
00000216	Rubén Álvarez Blanco	ruben.alvarez.216@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000046	Marta Álvarez Navarro	marta.alvarez.046@empresa.local	Recogida Sur - Equipo 1	Recogida de Residuos	Recogida Sur
00000180	Francisco Álvarez Riera	francisco.alvarez.180@empresa.local	Recogida Norte - Equipo 2	Recogida de Residuos	Recogida Norte
00000111	Raquel Álvarez Suárez	raquel.alvarez.111@empresa.local	Recogida Sur - Equipo 1	Recogida de Residuos	Recogida Sur
00000048	Paula Álvarez Torres	paula.alvarez.048@empresa.local	Recogida Sur - Equipo 2	Recogida de Residuos	Recogida Sur
00000069	Ana Bauzá Castro	ana.bauza.069@empresa.local	Recogida Sur - Equipo 1	Recogida de Residuos	Recogida Sur
00000042	Jorge Bauzá Gómez	jorge.bauza.042@empresa.local	Mant. Eléctrico - Guardia	Mantenimiento y Planta	Mantenimiento Eléctrico
00000213	Antonio Blanco Marín	antonio.blanco.213@empresa.local	Recogida Sur - Equipo 2	Recogida de Residuos	Recogida Sur

Flask API Connected

Figura 16. Directorio de personal visto por RRHH.

TR Tirme RRHH
Cuadro de Mando

USUARIO CONECTADO
atorres@empresa.local
Recursos Humanos (RR)

Directorio de personal

Empleados activos visibles para tu perfil.

GRUPO: Todos | ÁREA: Todas | BUSCAR: garcía

LISTADO DE EMPLEADOS 10 de 200 empleados

Nº PERSONAL	NOMBRE	CORREO	GRUPO	ÁREA	DEPARTAMENTO
00000026	Marta García Fernández	marta.garcia.026@empresa.local	Mant. Mecánico - Guardia	Mantenimiento y Planta	Mantenimiento Mecánico
00000099	Carmen García González	carmen.garcia.099@empresa.local	Flota - Taller	Logística y Transporte	Flota y Talleres
00000002	Maria García López	mgarcia@empresa.local	Grupo Técnico A	Desarrollo Software	Sistemas e IT
00000013	Jorge García Martínez	jorge.garcia.013@empresa.local	Flota - Conductores	Logística y Transporte	Flota y Talleres
00000234	Jorge García Ramírez	jorge.garcia.234@empresa.local	Recogida Norte - Equipo 2	Recogida de Residuos	Recogida Norte
00000206	Silvia García Rodríguez	silvia.garcia.206@empresa.local	Grupo Operativo B	Mantenimiento y Planta	Operaciones
00000210	David Gómez García	david.gomez.210@empresa.local	Valorización - Turno A	Tratamiento y Valorización	Valorización Energética
00000192	Alberto González García	alberto.gonzalez.192@empresa.local	Recogida Norte - Equipo 2	Recogida de Residuos	Recogida Norte
00000243	Maria Gutiérrez García	maria.gutierrez.243@empresa.local	Compostaje - Equipo 2	Tratamiento y Valorización	Compostaje
00000025	Luis López García	luis.lopez.025@empresa.local	Mant. Mecánico - Guardia	Mantenimiento y Planta	Mantenimiento Mecánico

Flask API Connected

Figura 17. Búsqueda de empleados por texto.

TR Tirme RRHH
Cuadro de Mando

USUARIO CONECTADO
rafael.coll.105@empresa.local
Mando (VB / FI)

Directorio de personal

Empleados activos visibles para tu perfil.

GRUPO: Todos | ÁREA: Todas | BUSCAR: Nombre, nº o correo

LISTADO DE EMPLEADOS 18 de 18 empleados

Nº PERSONAL	NOMBRE	CORREO	GRUPO	ÁREA	DEPARTAMENTO
00000216	Rubén Álvarez Blanco	ruben.alvarez.216@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000105	Rafael Coll Gutiérrez	rafael.coll.105@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000165	Margalida Delgado Pérez	margalida.delgado.165@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000176	Rafael Delgado Pérez	rafael.delgado.176@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000233	Paula Domínguez Riera	paula.dominguez.233@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000163	Javier Gil Martínez	javier.gil.163@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000059	Luis Gómez Coll	luis.gomez.059@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000035	Daniel Gutiérrez Bauzá	daniel.gutierrez.035@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000223	Sergio Iglesias Ortiz	sergio.iglesias.223@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000159	Juan Marín González	juan.marin.159@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000056	Marta Martín Suárez	marta.martin.056@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000193	Patricia Martínez Blanco	patricia.martinez.193@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte
00000065	Manuel Oliver Marín	manuel.oliver.065@empresa.local	Recogida Norte - Equipo 1	Recogida de Residuos	Recogida Norte

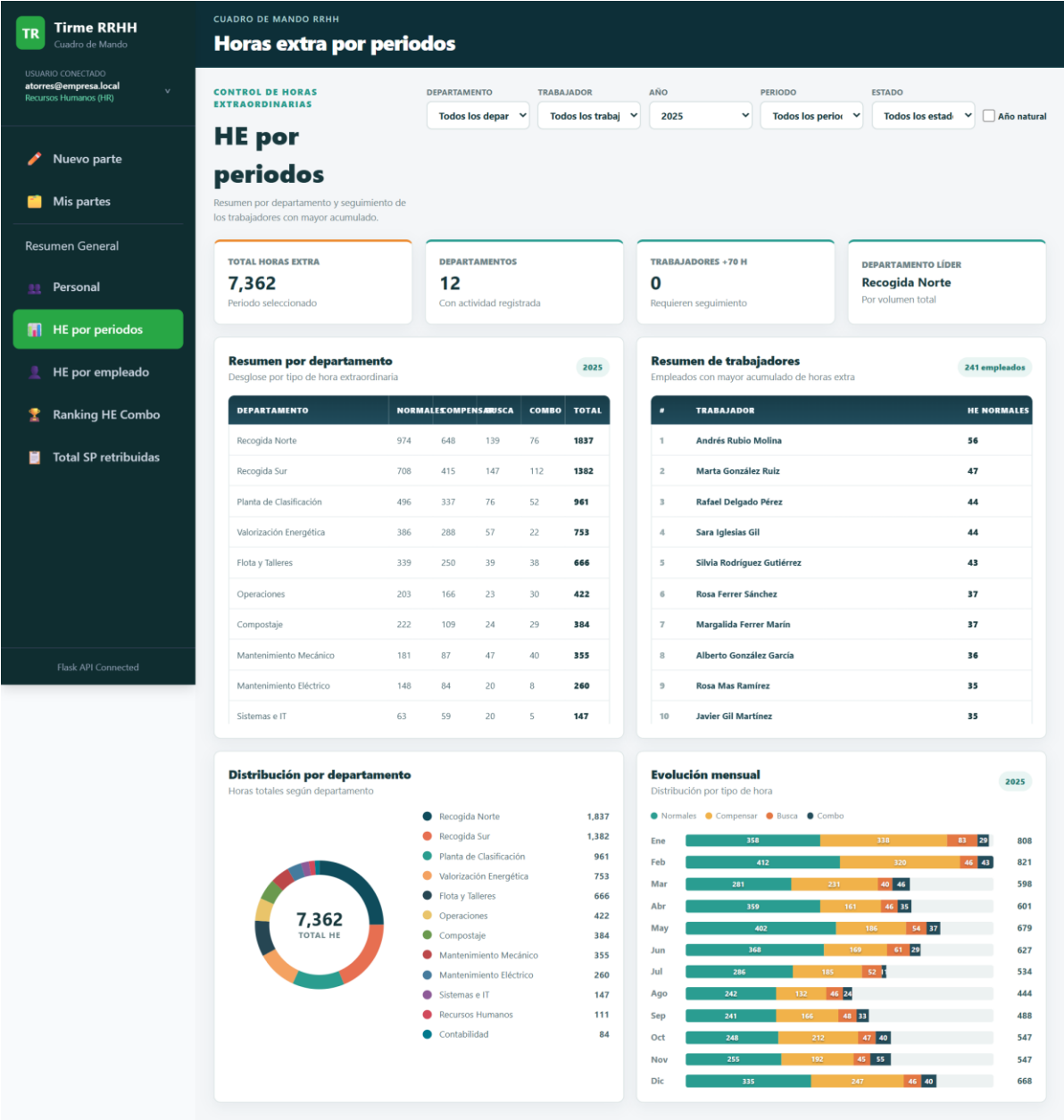
Flask API Connected

Figura 18. El mismo directorio visto por un mando: solo su grupo.



RF-06. Horas extra por periodos

- Filtros por año, periodo de nómina, departamento, trabajador, estado y la opción «año natural».
- Por periodo de nómina se agrupa por el mes de nómina (MES); en año natural, por la fecha real del parte.
- Indicadores (total de horas extra, departamentos, trabajadores con más de 70 h, departamento líder), desglose por departamento y por trabajador (normales, compensar, busca, combo...) y gráficos de distribución y evolución mensual.
- Los años del filtro y los estados salen de la base de datos.



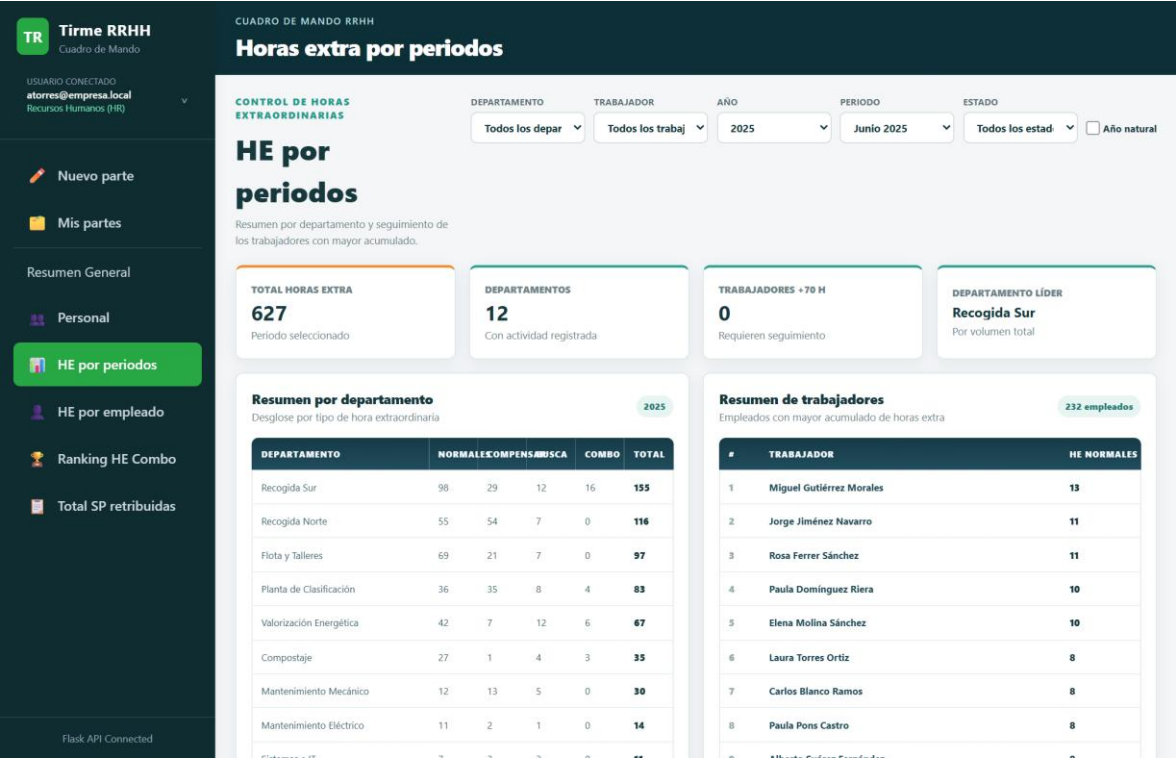


Figura 20. Filtrado por un periodo de nómina concreto.

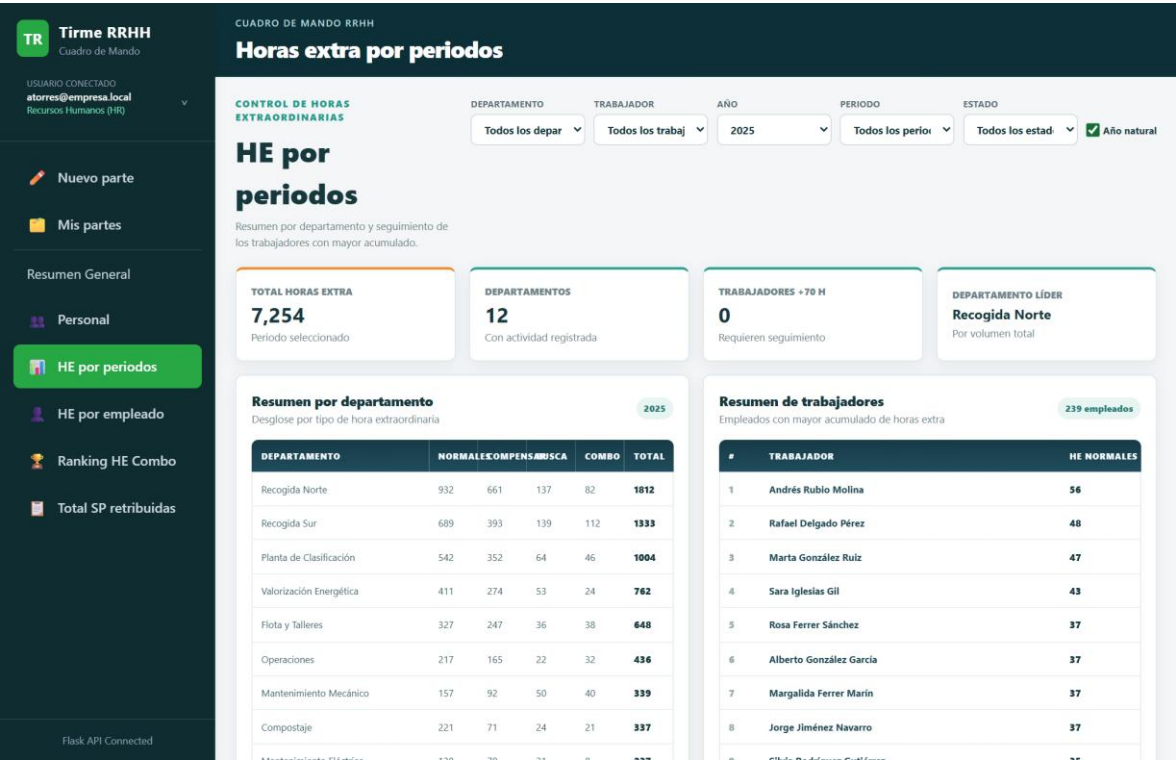
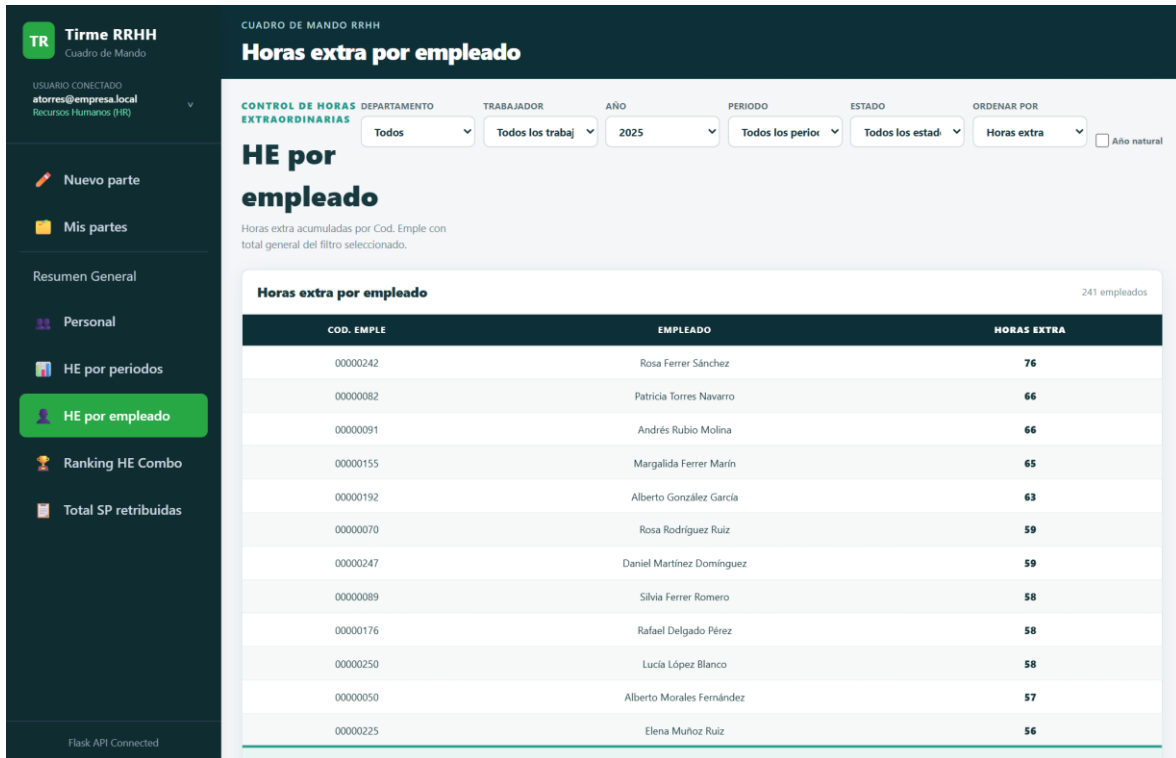


Figura 21. Modo «año natural»; se agrupa por la fecha del parte.

RF-07. Horas extra por empleado

- Total de horas extra de cada trabajador en el periodo elegido, con el total general.
- Orden por horas o por número de personal y los mismos filtros que la pantalla anterior.



COD. EMPL	EMPLEADO	HORAS EXTRA
00000242	Rosa Ferrer Sánchez	76
00000082	Patricia Torres Navarro	66
00000091	Andrés Rubio Molina	66
00000155	Margalida Ferrer Marín	65
00000192	Alberto González García	63
00000070	Rosa Rodríguez Ruiz	59
00000247	Daniel Martínez Domínguez	59
00000089	Silvia Ferrer Romero	58
00000176	Rafael Delgado Pérez	58
00000250	Lucía López Blanco	58
00000050	Alberto Morales Fernández	57
00000225	Elena Muñoz Ruiz	56

Figura 22. Horas extra por empleado (ejercicio 2025).

RF-08. Ranking de horas combo

- Por trabajador: horas combo programadas, total de horas extra, horas compensables y su conversión a horas de descanso (diurna laborable $\times 1,6$; diurna festiva y nocturna laborable $\times 2$; nocturna festiva $\times 2,5$).
- Totales del periodo. Solo para RRHH y mandos.

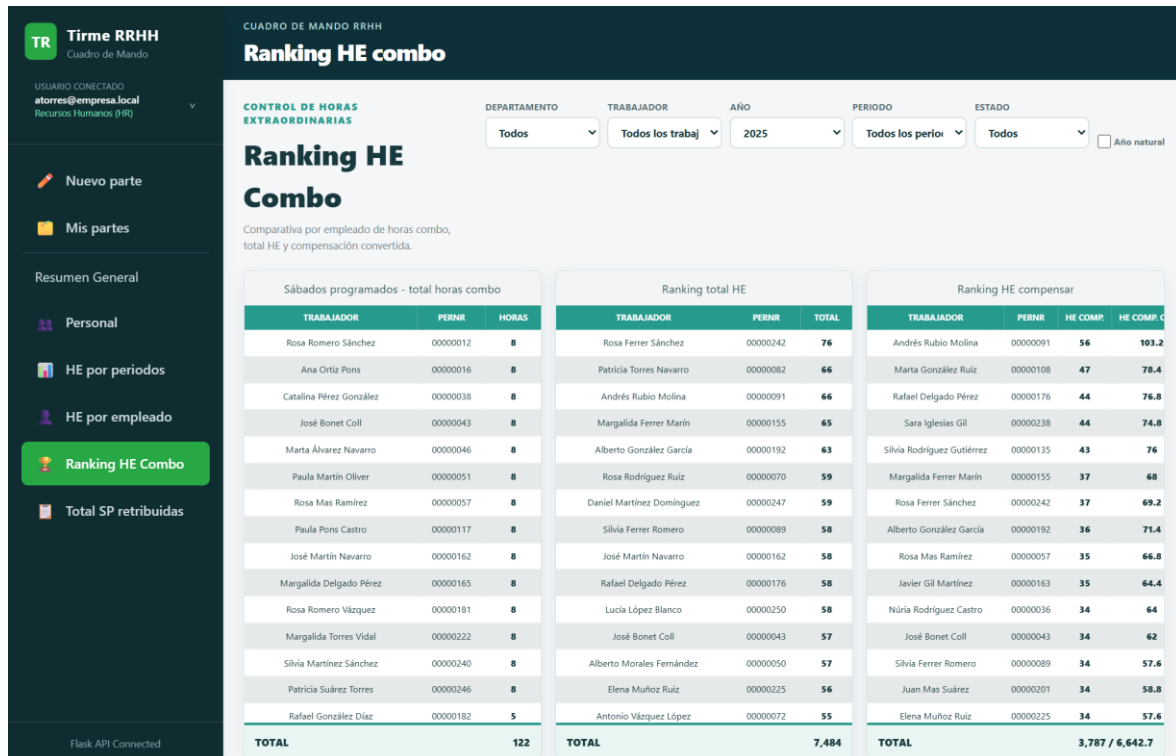


Figura 23. Ranking de horas combo y compensables.

RF-09. Total de servicios prestados (SP)

- Cuenta los partes con la marca SP activa ('X') en los últimos 12 y 24 meses, por persona y por departamento.
- Se puede elegir la fecha final de la ventana.



Figura 24. Servicios prestados de los últimos 12 y 24 meses.

RF-10. Formulario de parte de trabajo

Disponible para todos los perfiles; cada usuario registra **sus** partes. El formulario se construye a partir de una definición que sirve la API, de forma que las etiquetas y los bloques se cambian en un único fichero (`services/parte_campos.py`). Se organiza en cinco bloques:

1. **Día:** fecha y turno; el tipo de día (laborable, sábado, domingo o festivo) se calcula solo y se puede marcar «festivo local».
2. **Horario:** entrada, salida y pausa; la presencia se calcula en vivo y admite turnos que cruzan la medianoche.
3. **Horas extra y llamadas:** una tarjeta por tipo (normales, a compensar, busca, busca no pagada, combo, combo programadas y F) con sus cuatro tramos, y un bloque de llamadas (número de llamadas, que no suma horas).
4. **Situación del día y marcas:** Art. 21 descanso, Descanso o Teletrabajo (solo una) y las marcas en casillas (SP = superior categoría, SUST = sustitución...).
5. **Otros:** kilómetros y observaciones.

Controles lógicos, comprobados en el navegador y de nuevo en el servidor:



N.º	Regla
1	Fecha obligatoria, no posterior a hoy, y su periodo de nómina no puede estar traspasado.
2	Turno obligatorio (M, T, N o D).
3	Un solo parte por empleado, día y turno.
4	Horas enteras entre 0 y 24 por casilla; llamadas entre 0 y 20.
5	El total de horas extra no puede superar 24 h (las llamadas no suman).
6	Cada bloque con horas exige su motivo (máximo 150 caracteres).
7	Las horas festivas solo en sábado, domingo o festivo (o «festivo local»).
8	Entrada y salida en formato HH:MM; la pausa no puede igualar la jornada.
9	Si hay horas extra, la presencia es obligatoria y las horas extra no pueden superarla.
10	Como máximo una situación del día.
11	En teletrabajo no se admiten kilómetros.
12	La marca de sustitución exige el motivo de la sustitución.
13	Observaciones de 150 caracteres como máximo.

TR

Tirme RRHH

Cuadro de Mando

USUARIO CONECTADO
jpena@empresa.local
Empleado

Nuevo parte

Mis partes

HE por periodos

HE por empleado

Total SP retribuidas

Flask API Connected

CUADRO DE MANDO RRHH

Nuevo parte de trabajo

1. Día del parte

FECHA
02/10/2026

TURNO
Tarde

☐ Festivo local

Tipo de día: Laborable

2. Horario de trabajo

El tiempo de presencia se calcula a partir de la entrada, la salida y la pausa.

ENTRADA
07:00

SALIDA
18:30

PAUSA
00:30

PRESENCIA
11 h 00 min

3. Horas extra y llamadas

Horas extra normales

20 h

Diurna laborable

Diurna festiva

20

Nocturna laborable

Nocturna festiva

Motivo
(obligatorio)

Indica el motivo de horas extra normales.

Horas extra a compensar

5 h

Diurna laborable

Diurna festiva

5

Nocturna laborable

Nocturna festiva

Motivo
(obligatorio)

Indica el motivo de horas extra a compensar.

Horas de busca

0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas de busca no pagada

0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas combo

0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas combo programadas

0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas F

0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Llamadas (número)

0 llamadas (no suma)

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

4. Situación del día (solo una)

☒ Ninguna ☐ Art. 21 descanso ☐ Descanso ☐ Teletrabajo

Marcas

☐ SP - Superior categoría

☐ SUST - Sustitución

☐ CP

☐ PP

☐ B

☐ D

☐ DD

☐ P52

☐ P60

☐ BLV

☐ BSDF

☐ CPT

☐ PN

5. Otros datos

KILÓMETROS

OBSERVACIONES

Nuevo parte

HORAS EXTRA
25 / 24 h

El total de horas extra (25 h) supera el máximo de 24 h.

PRESENCIA
11 h 00 min

- Indica el motivo de horas extra normales.
- Indica el motivo de horas extra a compensar.
- El total de horas extra (25 h) supera el máximo de 24 h.

Volver

Guardar

Figura 25. Formulario con 25 h extra y sin motivos: total en rojo, mensajes junto a cada campo y casillas festivas bloqueadas en día laborable.

CUADRO DE MANDO RRHH

Nuevo parte de trabajo

1. Día del parte

FECHA
02/10/2026

TURNO
Tarde

☐ Festivo local

Tipo de día: Laborable

2. Horario de trabajo

El tiempo de presencia se calcula a partir de la entrada, la salida y la pausa.

ENTRADA
07:00

SALIDA
18:30

PAUSA
00:30

PRESENCIA
11 h 00 min

3. Horas extra y llamadas

Horas extra normales
3 h

Diurna laborable
3

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo
(obligatorio)
Avería en la línea de clasificación

Horas extra a compensar
0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas de busca
0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas de busca no pagada
0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas combo
0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas combo programadas
0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Horas F
0 h

Diurna laborable

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo

Llamadas (número)
2 llamadas (no suma)

Diurna laborable
2

Diurna festiva

Nocturna laborable

Nocturna festiva

Motivo
(obligatorio)
Guardia telefónica

4. Situación del día (solo una)

☒ Ninguna ☐ Art. 21 descanso ☐ Descanso ☐ Teletrabajo

Marcas

☒ SP - Superior categoría

☐ SUST - Sustitución

☐ CP

☐ PP

☐ B

☐ D

☐ DD

☐ P52

☐ P60

☐ BLV

☐ BSDF

☐ CPT

☐ PN

5. Otros datos

KILÓMETROS

OBSERVACIONES

Nuevo parte

HORAS EXTRA
3 / 24 h

PRESENCIA
11 h 00 min

Volver

Guardar

TR Tirme RRHH

Cuadro de Mando

USUARIO CONECTADO
jperez@empresa.local
Empleado

Nuevo parte

Mis partes

HE por periodos

HE por empleado

Total SP retribuidas

Flask-API Connected

Figura 26. Formulario completo y válido: presencia calculada (11 h), horas con su motivo y marca SP.

```
Validación en el servidor: POST /api/mis-partes con 25 h y sin motivo

$ curl -X POST /api/mis-partes
HTTP/1.1 400
{
  "message": "Revisa los datos del parte.",
  "errores": {
    "motivos.compensar": "Indica el motivo de: horas extra a compensar.",
    "total_he": "El total de horas extra (25 h) supera el máximo de 24 h."
  }
}
```

Figura 27. El servidor rechaza el mismo parte aunque se envíe sin pasar por el formulario.

RF-11. Mis partes

- Lista los partes propios del ejercicio elegido con estado (y su color), presencia, horas extra y llamadas.
- Los partes en estado B (creados por el empleado, sin visto bueno) se pueden editar y borrar; los demás aparecen con un candado y solo se pueden consultar.
- El servidor calcula los campos que no vienen del navegador: número de personal (del token), periodo de nómina (de zperiodos), grupo, departamento, presencia y estado.

TR Tírme RRHH
Cuadro de Mando

USUARIO CONECTADO
jperez@empresa.local
Empleado

Nuevo parte

Mis partes

HE por periodos

HE por empleado

Total SP retribuidas

Flask API Connected

CUADRO DE MANDO RRHH

Mis partes de trabajo

Mis partes de trabajo
Puedes modificar o borrar un parte hasta que tenga el visto bueno del jefe de área.

EJERCICIO
2026 + Nuevo parte

PARTES DEL EJERCICIO 2026 109 partes

FECHA	TURNO	ESTADO	PRESENCIA	HORAS EXTRA	LLAMADAS	ACCIONES
02/10/2026	L	● Creado por el empleado	11:00	3	2	Editar Borrar
01/10/2026	L	● Creado por el encargado	8:00	0	0	Ver
30/09/2026	L	● Creado por el encargado	0:00	0	0	Ver
29/09/2026	L	● Visto bueno por el jefe de área	0:00	1	0	Ver
28/09/2026	L	● Visto bueno por el jefe de área	8:00	0	0	Ver
25/09/2026	L	● Modificado y visto bueno por el jefe de área	0:00	0	0	Ver
18/09/2026	L	● Creado por el encargado	0:00	0	0	Ver
17/09/2026	L	● Creado por el empleado	0:00	0	0	Editar Borrar
15/09/2026	L	● Creado por el empleado	0:00	7	0	Editar Borrar
14/09/2026	L	● Creado por el empleado	0:00	0	0	Editar Borrar
11/09/2026	L	● Firmado por el jefe de departamento	0:00	0	0	Ver
09/09/2026	L	● Traspasado a nómina por RRHH	8:00	0	0	Ver
07/09/2026	L	● Traspasado a nómina por RRHH	0:00	4	0	Ver
31/08/2026	L	● Traspasado a nómina por RRHH	8:00	0	0	Ver

Figura 28. Mis partes: el parte recién guardado (editable) y los ya validados (solo consulta).

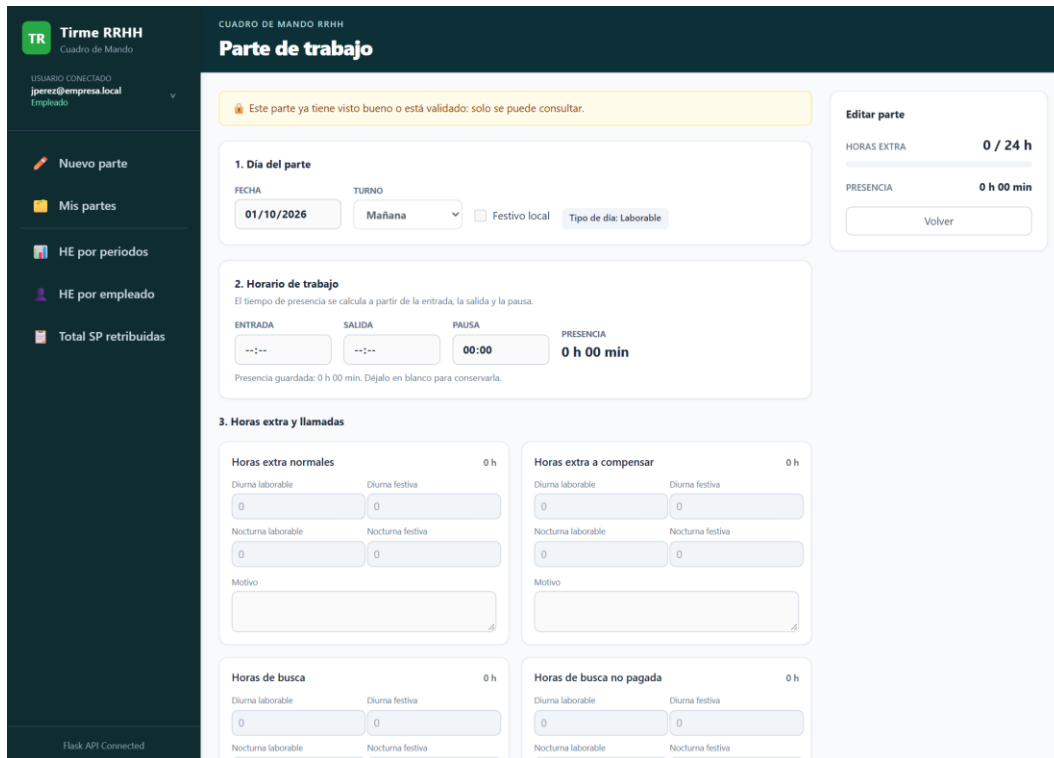


Figura 29. Un parte ya validado se abre en modo consulta.

RF-12. Persistencia en la base de datos

Al guardar, el parte se escribe en la tabla `zparte` con el mismo formato que usa la aplicación Java, por lo que aparece inmediatamente en el circuito de firmas y en el resto de pantallas del dashboard.

```
Fila guardada en zparte tras pulsar "Guardar" en el formulario

MariaDB [epartes_local]> SELECT MANDT, PERNR, PADAT, ..., USER_ID FROM zparte WHERE MANDT =
1791149096415\G
***** 1. row *****
      MANDT: 1791149096415
      PERNR: 00000001
      PADAT: 2026-10-02
      TIPO: T
      TURNO: T
      MES: 10
      EJERC: 2026
      STAT: B
      CODGR: 1
      DPTO: Sistemas e IT
      TIPODIA: L
      HN: 11
      HNDEC: 110000
      DL: 3
      LLDL: 2
      MOTIVOHE: Avería en la línea de clasificación
      MOTIVOLLA: Guardia telefónica
      SP: X
      TELETRABAJO: f
      DESCANSO: f
      ART21DESC: f
      PERNRCR: 00000001
      CRDAT: 2026-10-04
      USER_ID: 00000001
```

Figura 30. Fila creada en zparte por el formulario: periodo de nómina, estado B, presencia 110000, horas, motivos y marcas.

RF-13. API REST documentada

- Tres grupos de endpoints: autenticación (/api/login, /api/get-roles, /api/me), partes y cuadro de mando (/api/partes/...) y partes del propio empleado (/api/mis-partes/...), 20 en total.
- Documentación interactiva en /api/doc con el botón «Authorize» para probar con un token.

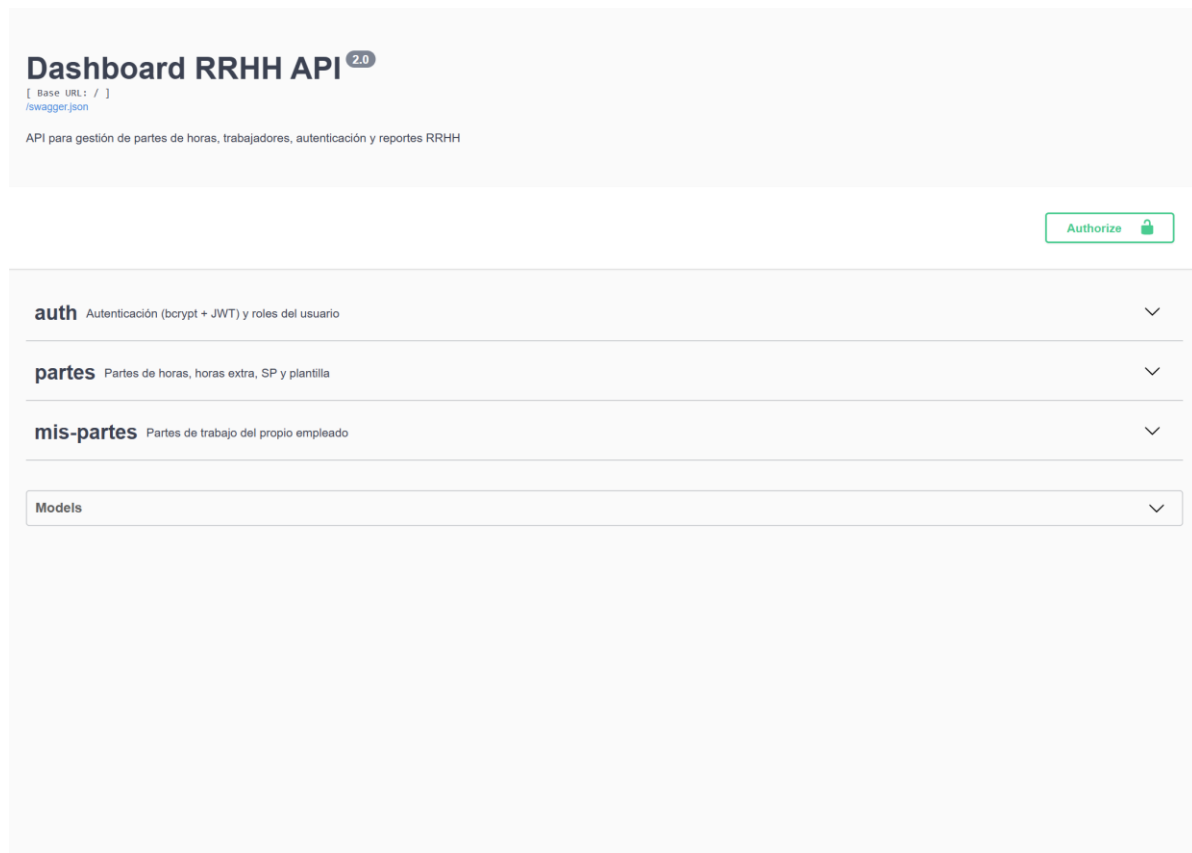


Figura 31. Documentación Swagger de la API.

5.2 Requisitos no funcionales

RNF-01. Seguridad

- Contraseñas con hash bcrypt; las que estuvieran en texto plano no se aceptan.
- Mismo mensaje y tiempo de respuesta tanto si el correo no existe como si la contraseña es errónea.
- Todos los endpoints de datos exigen token (401) y perfil autorizado (403); el número de personal sale siempre del token, nunca del navegador.
- Secretos fuera del repositorio (.env ignorado por Git; solo se versiona .env.example).
- CORS limitado al frontend y servidor Flask escuchando solo en local.

```
Pruebas de seguridad contra la API (peticiones reales)

$ curl -X POST /api/login
HTTP 401
{
  "message": "Correo o contraseña incorrectos."
}

$ curl -X POST /api/get-roles
HTTP 200
{
  "success": true,
  "roles": [
    "hr",
    "mando",
    "empleado"
  ]
}

$ curl -X GET /api/partes
HTTP 401
{
  "message": "Debes iniciar sesión."
}

$ curl -X GET /api/partes/plantilla-resumen
HTTP 403
{
  "message": "Tu perfil no tiene acceso a este recurso."
}

$ curl -X GET /api/partes/empleado/00000003
HTTP 404
{
  "message": "No se encontraron partes para el empleado 00000003"
}
```

Figura 32. Pruebas reales contra la API: credenciales erróneas (401), sin token (401), perfil sin permiso (403) y parte de otro empleado (404).

```
Contraseñas guardadas como hash bcrypt (nunca en texto plano)

MariaDB [epartes_local]> SELECT NUMPER, NAME, LEFT(PASSWORD, 29), ACTIVE FROM userpayroll ...
NUMPER NAME PASSWORD_hash_bcrypt ACTIVE
00000001 Juan $2b$12$CKXaZ0hM2Fb0tTr3u0ZEN. 1
00000002 María $2b$12$CKXaZ0hM2Fb0tTr3u0ZEN. 1
00000003 Carlos $2b$12$CKXaZ0hM2Fb0tTr3u0ZEN. 1
00000004 Ana $2b$12$CKXaZ0hM2Fb0tTr3u0ZEN. 1
00000005 Elena $2b$12$CKXaZ0hM2Fb0tTr3u0ZEN. 1
```

Figura 33. Contraseñas guardadas como hash bcrypt. En los datos de demostración todos los usuarios comparten la misma contraseña, por eso coincide el hash.

RNF-02. Rendimiento

Con unos 100.000 partes, todas las pantallas responden en menos de 0,35 s. Las consultas de un empleado o de un periodo concreto pasan de recorrer la tabla completa a leer solo sus filas gracias a los índices (por ejemplo, de 210 ms a 7 ms). Los agregados anuales de RRHH, que leen un tercio de la tabla, usan un recorrido completo, que es el plan óptimo en ese caso (detalle en el apartado 4.5 y en docs/rendimiento.md).

RNF-03. Calidad y mantenibilidad

- **Programación orientada a objetos:** servicios como clases con una responsabilidad cada una (RolService, AuthService, HorasExtraService, PlantillaService, SpService, ParteService, ParteValidator, FiltroPartesBuilder...) y modelos ORM.
- Rutas finas: solo leen parámetros, comprueban el perfil y delegan en los servicios.
- **191 tests automáticos:** 159 en el backend (login, aislamiento de datos por perfil, cálculos de horas extra con valores conocidos, formulario, generador de datos) y 32 en el frontend.

```
Tests automáticos del backend (pytest)

(venv) $ python -m pytest -v
tests/test_acceso_datos.py::test_listado_de_partes[atorres-hr] PASSED
tests/test_acceso_datos.py::test_listado_de_partes[mgarcia-mando] PASSED
tests/test_acceso_datos.py::test_listado_de_partes[crodriguez-mando] PASSED
tests/test_acceso_datos.py::test_listado_de_partes[jperez-empleado] PASSED
tests/test_acceso_datos.py::test_listado_de_partes[eruiiz-hr] PASSED
tests/test_acceso_datos.py::test_listado_de_partes[eruiiz-mando] PASSED
tests/test_acceso_datos.py::test_listado_de_partes[mgarcia-empleado] PASSED
tests/test_acceso_datos.py::test_he_por_empleado[atorres-hr] PASSED
tests/test_acceso_datos.py::test_he_por_empleado[mgarcia-mando] PASSED
tests/test_acceso_datos.py::test_he_por_empleado[crodriguez-mando] PASSED
tests/test_acceso_datos.py::test_he_por_empleado[jperez-empleado] PASSED
tests/test_acceso_datos.py::test_he_por_empleado[eruiiz-hr] PASSED
... (147 tests más)

Tests por fichero:
 34 tests/test_acceso_datos.py
 14 tests/test_auth.py
 19 tests/test_calculos_he.py
 20 tests/test_filtro_partes.py
  9 tests/test_generador.py
 11 tests/test_mis_partes.py
 30 tests/test_parte_validator.py
 12 tests/test_personal.py
  6 tests/test_rol_service.py
  4 tests/test_smoke.py

===== 159 passed in 40.26s =====
```

Figura 34. Ejecución de los tests del backend: 159 pasados.

```
Tests automáticos del frontend (Angular + Vitest)

$ npx ng test --watch=false
permisos.spec.ts · parte-validacion.spec.ts · he.service.spec.ts · auth.service.spec.ts
auth.interceptor.spec.ts · header-filtros.spec.ts · sidebar.spec.ts · app.spec.ts
Test Files  8 passed (8)
Tests      32 passed (32)
Start at   23:23:26
Duration   11.19s (transform 1.78s, setup 11.78s, import 2.10s, tests 614ms, environment 45.23s)
```

Figura 35. Ejecución de los tests del frontend: 32 pasados.

RNF-04. Portabilidad e instalación

La base de datos se levanta con Docker Compose y se carga sola con el volcado; el backend y el frontend se instalan con `pip` y `npm`. El proceso funciona igual en Windows y Linux (apartado 6).



RNF-05. Usabilidad

Menú adaptado al perfil, título de pantalla en la cabecera, filtros con las opciones reales de la base de datos, mensajes de error junto al campo afectado y resumen fijo en el formulario (total de horas sobre 24 con una barra que se vuelve roja al superarlo).

RNF-06. Protección de datos

Ningún dato real en el código, en los datos de demostración ni en esta memoria. El directorio de personal no expone datos sensibles y cada perfil solo accede a la información que le corresponde.



6. MANUAL DE INSTALACIÓN

Este manual parte de un equipo sin nada instalado. Todos los comandos se han probado; donde difieren, se indica la versión para **Windows (PowerShell)** y para **Linux (bash)**.

6.1 Requisitos previos

Software	Versión	Comprobación
Python	3.10 o superior (probado con 3.13)	python --version (Linux: python3 --version)
Node.js	22.22 o superior, o 24.15 o superior	node --version
Docker y Docker Compose v2	Cualquier versión reciente (Docker Desktop en Windows)	docker compose version
Git (opcional)	—	git --version

En Windows, al instalar Python hay que marcar «Add python.exe to PATH», y Docker Desktop debe estar abierto. En Debian/Ubuntu: `sudo apt install -y python3 python3-venv python3-pip git`, Docker Engine según la guía oficial y Node.js 22/24. El puerto **3306** debe estar libre.

6.2 Obtener el proyecto y crear los ficheros .env

Se descomprime el zip de la entrega (o se clona el repositorio) y se trabaja desde la carpeta raíz `tirme-rrhh`. Hay dos ficheros de configuración: uno para Docker en la raíz y otro para Flask en `dashboard_rrhh`. Se crean copiando las plantillas:

```
# Windows (PowerShell)
Copy-Item .env.example .env
Copy-Item dashboard_rrhh\.env.example dashboard_rrhh\.env

# Linux
cp .env.example .env
cp dashboard_rrhh/.env.example dashboard_rrhh/.env
```

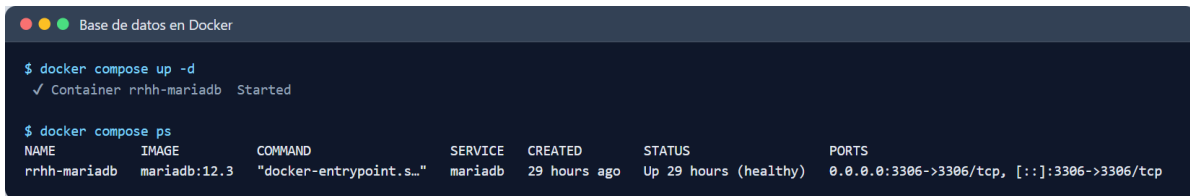
En ambos ficheros se cambian las contraseñas de ejemplo: **DB_USER**, **DB_PASSWORD**, **DB_NAME** y **DB_PORT** deben coincidir. En `dashboard_rrhh/.env` se generan también **SECRET_KEY** y **JWT_SECRET_KEY**, de al menos 32 caracteres:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

6.3 Arrancar la base de datos

```
docker compose up -d
docker compose ps      # esperar a que el estado sea "healthy"
```

La primera vez, MariaDB carga automáticamente `epartes_local_backup.sql` y crea la base de datos vacía `epartes_test` que usan los tests.



```
$ docker compose up -d
✓ Container rrhh-mariadb Started

$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
rrhh-mariadb	mariadb:12.3	"docker-entrypoint.s..."	mariadb	29 hours ago	Up 29 hours (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp

Figura 36. Contenedor de MariaDB en marcha y saludable.

6.4 Backend (Flask)

```
# Windows (PowerShell)
cd dashboard_rrhh
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python seed.py          # usuarios de prueba
python generar_datos.py  # recomendado: datos de demostración (≈ 25 s)
python -m pytest         # 159 passed
python app.py

# Linux
cd dashboard_rrhh
python3 -m venv venv
source venv/bin/activate
python -m pip install -r requirements.txt
python seed.py
python generar_datos.py
python -m pytest
python app.py
```

El paso `python generar_datos.py` es **recomendado**. Con el volcado y los usuarios de prueba solo hay 5 empleados y 3 partes, por lo que las pantallas del cuadro de mando (horas extra, ranking, servicios prestados y sus gráficos) aparecerían casi vacías. El script carga datos inventados (250 empleados en 25 grupos y unos 100.000 partes) con los que la aplicación se ve como en las capturas de esta memoria: se aprecia qué ve cada perfil y el rendimiento se puede medir con un volumen realista. Mantiene los usuarios de prueba y no afecta a los tests, que usan su propia base de datos (`epartes_test`).



Si PowerShell bloquea `Activate.ps1`, se ejecuta una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`. El servidor queda en `http://localhost:5000` (documentación en `http://localhost:5000/api/doc`).

6.5 Frontend (Angular)

```
cd frontend
npm ci
npm start
```

Se abre `http://localhost:4200` y se entra con uno de los usuarios de prueba (contraseña `Tirme2026!`), por ejemplo `atorres@empresa.local` (RRHH) o `jperez@empresa.local` (empleado).

6.6 Operaciones habituales y problemas frecuentes

Situación	Solución
Volver a cargar la base de datos desde cero	<code>docker compose down -v</code> , después <code>docker compose up -d</code> y <code>python seed.py</code>
«port is already allocated» al arrancar Docker	El puerto 3306 está ocupado: parar el otro servidor o cambiar <code>DB_PORT</code> en los dos <code>.env</code> .
Flask: «Access denied for user»	Las credenciales de los dos <code>.env</code> no coinciden (o se cambiaron tras crear el volumen: recrearlo).
Login: «Correo o contraseña incorrectos» con un usuario de prueba	Falta ejecutar <code>python seed.py</code> .
pytest muestra tests «skipped»	MariaDB no está levantada o falta la base de datos <code>epartes_test</code> (recrear el volumen).
La página de login muestra una versión antigua	Recargar sin caché (Ctrl + F5).
Medir el rendimiento de las consultas	<code>python medir_rendimiento.py</code> (≈ 2 min; resultados en <code>docs/rendimiento.md</code>).



7. METODOLOGÍA Y HERRAMIENTAS

7.1 Forma de trabajo

El proyecto se desarrolló por fases, cada una con un objetivo concreto y verificable: instalación reproducible, autenticación y control de accesos, refactorización del backend en servicios, frontend con datos reales, datos de demostración y medición de rendimiento, formulario de partes y, por último, esta documentación. Antes de cada fase se redactaba un plan con las decisiones abiertas, se revisaba y solo después se implementaba. Cada fase terminaba con los tests en verde, una prueba de la aplicación en el navegador y un commit en Git.

7.2 Uso de un asistente de inteligencia artificial

Durante el desarrollo utilicé **Claude Code**, un asistente de programación basado en IA, como herramienta de apoyo: para proponer planes de trabajo, generar y revisar código, escribir tests, ejecutar comprobaciones y redactar documentación. Lo declaro aquí de forma expresa, igual que se declararía cualquier otra herramienta del stack.

Las decisiones del proyecto fueron más y quedaron reflejadas en cada plan antes de implementarse. Entre ellas:

- El alcance funcional y los perfiles a partir de la propuesta aprobada, con la regla de que el filtrado de datos se aplica siempre en el servidor.
- Qué pantallas mantener y cuáles eliminar (por ejemplo, convertir «Personal» en un directorio real y eliminar «Turnos», que mostraba datos inventados).
- El alcance del formulario de parte: cálculo de la presencia por hora de entrada y salida, fechas permitidas, edición solo en estado B y etiquetas de las marcas.
- No usar datos reales por la LOPD y limitar su uso a estadísticas agregadas.
- Rotar las claves al detectar que el historial público del repositorio contenía contraseñas antiguas.
- Medir el rendimiento con tres escenarios de índices y no modificar el esquema compartido.

Todo lo generado se verificó antes de darlo por bueno: con los tests automáticos (191 en total), con recorridos completos de la aplicación en el navegador para cada perfil y revisando cada cambio. Varias propuestas se corrigieron durante esa verificación (por ejemplo, un fallo de detección de cambios en el login de Angular o tests que compartían una sesión de base de datos y leían datos antiguos).



7.3 Herramientas

Herramienta	Uso
Visual Studio Code	Edición del código y de la documentación.
Git y GitHub	Control de versiones por fases y repositorio público.
Docker Desktop	Base de datos MariaDB local y base de datos de tests.
Swagger UI	Pruebas manuales de la API.
Playwright (con Microsoft Edge)	Recorridos automáticos en el navegador y capturas de esta memoria.
diagrams.net	Diagrama del modelo de datos (docs/modelo_datos.drawio).
Claude Code	Asistente de IA de apoyo, según lo descrito en el apartado 7.2.



8. CONCLUSIONES

8.1 Conclusiones generales

El proyecto cumple los requisitos mínimos del curso (base de datos, programación orientada a objetos, uso de un framework, login y control de accesos, y documentación) y la mayor parte de la propuesta aprobada. El resultado es una aplicación que funciona sobre el esquema real de la empresa sin modificarlo, que separa lo que ve cada perfil en el servidor y que permite a cualquier empleado registrar su parte con controles que antes no existían.

Trabajar sobre un esquema heredado ha sido la principal diferencia respecto a un proyecto creado desde cero: en lugar de diseñar las tablas a medida, hubo que entender columnas con nombres abreviados, deducir su significado a partir de los datos y adaptar el código a ese modelo.

8.2 Aprendizajes

- **Control de accesos en el servidor:** ocultar un menú no protege nada; la protección real está en filtrar cada consulta según el perfil y en comprobarlo con tests de aislamiento.
- **Autenticación:** hash de contraseñas con bcrypt, tokens JWT y cómo evitar revelar información en los mensajes de error.
- **Arquitectura:** separar rutas, servicios y modelos hace que las reglas (por ejemplo, las 13 del formulario) se puedan probar de forma aislada.
- **Rendimiento:** leer planes de ejecución (EXPLAIN), entender cuándo un índice ayuda y cuándo un recorrido completo es lo correcto, y medir antes de decidir.
- **Datos y LOPD:** generar datos ficticios realistas a partir de estadísticas agregadas, sin usar información personal.
- **Calidad:** escribir tests con valores calculados a mano antes de refactorizar permite cambiar el código con seguridad.

8.3 Dificultades encontradas

- Tablas con 102 columnas sin documentación: se resolvió con un análisis estadístico de los datos.
- Contraseñas antiguas en el historial público del repositorio: se eliminaron de los ficheros, se generaron claves nuevas y se recreó la base de datos.
- Tests que leían datos antiguos porque compartían una transacción de base de datos: se corrigió la configuración para que cada petición use su propia sesión, como en producción.



- Diferencias entre entornos (puerto ocupado por un contenedor antiguo, finales de línea de Windows en scripts de Linux), resueltas y documentadas en el manual.



9. EVOLUTIVOS DEL PROYECTO

La versión entregada cubre la consulta del cuadro de mando y el registro del parte. A continuación se describen las mejoras previstas para futuras versiones, agrupadas por tipo y con la prioridad que les asignaría en una siguiente iteración.

9.1 Mejoras funcionales

Circuito de firmas desde el dashboard (prioridad alta)

Hoy el visto bueno, la firma y el traspaso a nómina se hacen en la aplicación Java. El siguiente paso natural es que un mando VB pueda dar el visto bueno a los partes de su grupo, que un FI los firme y que RRHH los traspase, todo desde el dashboard. La base ya existe: el filtro por perfil limita qué partes ve cada mando y la tabla `zparte` tiene las columnas de cada firma (PERNRVB, VBDAT, PERNRFI, FIDAT...). Faltaría un servicio que valide las transiciones de estado permitidas (de A o B a C o D con el visto bueno, a E o F con la firma y a G, H o I en RRHH) y los plazos de cada periodo de `zperiodos`.

Indicadores de absentismo y rotación (prioridad media)

Completar el módulo de KPIs de la propuesta inicial: tasa de absentismo a partir de los partes de descanso y de las ausencias, y rotación a partir de las fechas de alta y baja de `userpayroll`, con su evolución mensual y comparativa entre departamentos.

Exportación de informes (prioridad media)

Descargar cada pantalla del cuadro de mando en Excel (para que RRHH pueda seguir trabajando los datos) y en PDF (para adjuntarlo a informes). Los datos ya llegan agregados desde la API, por lo que bastaría con un endpoint de exportación por pantalla que respete el mismo filtro de perfil.

Partes del equipo para los mandos (prioridad baja)

Permitir que un encargado registre partes en nombre de los miembros de su grupo, algo que la aplicación Java ya contempla (columna `PERNRCCR`, creador del parte). Exigiría comprobar en el servidor que el empleado pertenece a un grupo del mando.

9.2 Mejoras técnicas

Evolutivo	Descripción	Prioridad
Seguridad de la sesión	Token de refresco de corta duración, lista de tokens revocados al cerrar sesión y límite de intentos de inicio de sesión por usuario e IP.	Alta



Evolutivo	Descripción	Prioridad
Despliegue en producción	Servidor WSGI (waitress en Windows o gunicorn en Linux) detrás de un proxy inverso con HTTPS, y el frontend compilado servido como estático.	Media
Integración continua	Ejecutar los 191 tests en cada commit (por ejemplo, con GitHub Actions y un servicio de MariaDB).	Media
Índices propuestos	Aplicar (CODGR, PADAT) y (SP, PADAT) en producción tras validarlos con el equipo de la aplicación Java.	Media
Caché de agregados	Guardar en caché los totales de periodos ya traspasados a nómina, que no pueden cambiar.	Baja
Etiquetas de las marcas	Completar la descripción de las marcas cuyo significado no está documentado en la aplicación corporativa.	Baja

9.3 Hoja de ruta

Propongo abordar primero la seguridad de la sesión, porque es un requisito antes de publicar la aplicación fuera de la red local, y a continuación el circuito de firmas, que es la mejora que más trabajo manual ahorra a los mandos. Después vendrían el despliegue con integración continua y, por último, los nuevos indicadores y la exportación, que amplían el cuadro de mando sin cambiar su arquitectura.

El código fuente completo está disponible en <https://github.com/ramonmarquezfernandez-code/dashboard-rrhh-tirme>.